

OmegaSelf

Evidence-Grounded Self-Modeling for OmegaClaw

Architecture, underlying theory, technical design, and coding-agent deployment guide

Document status	Concrete architecture and implementation proposal, version 0.1
Target	OmegaClaw systems using PeTTa/MeTTa, Hyperon mechanisms, PLN/NAL-style uncertain inference, and Python or Rust runtime bridges
Core stance	The self-model is a continuously tested, defeasible theory about the running agent, not a privileged ego atom or stored autobiography
Date	15 July 2026
Companion artifact	Coding-agent pack containing schemas, a tested Python reference runtime, MeTTa scaffolding, ADRs, configs, tests, and integration notes

This document synthesizes the supplied OmegaSelf design, the directed-identity and governance-seam formalizations, the phenomenological identity note, and the current public OmegaClaw extension and dispatch architecture consulted on 15 July 2026. Structural claims, engineering decisions, hypotheses, and phenomenological interpretations are labeled separately.

Contents

Executive summary	viii
How to read this document	x
I Motivation and key ideas	1
1 Why OmegaClaw needs a self-modeling subsystem	2
1.1 The operational problem	2
1.2 Failure modes of naive self-modeling	2
1.2.1 Autobiography drift	2
1.2.2 Authority inversion	2
1.2.3 Context collapse	3
1.2.4 Confidence inflation	3
1.2.5 Copy-symmetry and false ownership	3
1.2.6 Continuity collapse	3
1.2.7 Detectors without consumers	3
1.2.8 Counterfactual contamination	3
1.2.9 Invisible governance	3
1.3 What success looks like	3
2 Core design stance and principles	5
2.1 The central decision	5
2.2 Ten key ideas	5
2.3 Non-goals	5
3 Non-negotiable invariants	7
3.1 Evidence and replay	7
3.2 Separation of inference and action	7
3.3 Indexical grounding	7
3.4 Forks and counterfactuals	7
3.5 Operational completeness	8
4 The minimum useful product	9
II Underlying theory	10
5 The self-model as a continuously tested theory	11

5.1	From description to prediction	11
5.2	Closed-loop formulation	11
5.3	Discrepancy as the learning signal	12
5.4	Active testing and value of information	12
6	Evidence semantics and provenance	13
6.1	Three temporal orders	13
6.2	Event sourcing	13
6.3	Evidence closure	14
6.4	Correlated evidence	14
7	Faceted self-theory and cross-facet coherence	15
7.1	Why facets are necessary	15
7.2	Disagreement is data	15
7.3	Coherence conditions	16
8	Indexical grounding: resolving SelfHereNow	17
8.1	The indexical gap	17
8.2	Bundle members	17
8.3	Resolution semantics	18
9	Continuity as directed transport	19
9.1	Why equality is the wrong primitive	19
9.2	Snapshots and transports	19
9.3	Endomorphism monoid and common fixed set	20
9.4	Path dependence and discrete holonomy	20
9.5	Finite core with lazy extension	20
10	PLN/NAL self-beliefs	22
10.1	Contextual truth values	22
10.2	Installed, reachable, authorized, effective	22
10.3	Priors, absence, and version changes	23
10.4	Confidence and action thresholds	23
10.5	Ordinal-graded consolidation as a research bridge	23
11	Governance seam and policy differential	24
11.1	Policy-parameterized revision	24
11.2	The admissible policy class is load-bearing	24
11.3	False positives and controls	25
11.4	Local decision seam	25
11.5	Governance versus slow epistemics	25
12	Attribution and social identity	26
12.1	Attribution is query-relative	26
12.2	Phenomenological interpretations and limits	26
III	Proposed technical design	27
13	System architecture and authority boundaries	28

13.1	Architectural overview	28
13.2	Five architectural planes	28
13.3	Logical AtomSpace organization	29
13.4	Authority hierarchy	30
13.5	Component responsibilities	30
14	Data model and semantic contracts	32
14.1	Identifier and versioning discipline	32
14.2	Observation envelope	32
14.3	Evidence closure	33
14.4	Contextual self-belief	34
14.5	Prediction and resolution	34
14.6	Typed proposal and policy decision	34
14.7	Runtime anchor	35
14.8	Snapshots and transports	35
14.9	Tripwire and consumer	36
15	OmegaClaw integration design	37
15.1	The causal seam in the loop	37
15.2	Turn lifecycle hooks	37
15.3	Bridge boundary	38
15.4	MeTTa module layout	39
15.5	Skill catalogue changes	39
15.6	Compact prompt interface	39
15.7	Concurrency and time-of-check/time-of-use	40
16	Self-belief inference and predictive calibration	41
16.1	Capability inference pipeline	41
16.2	Dependence-aware evidence aggregation	41
16.3	Context specialization and generalization	42
16.4	Staleness	42
16.5	Prediction scoring	42
16.6	Active probe scheduling	43
16.7	Commitment model	43
16.8	Epistemic self-model	43
16.9	Cross-facet coherence	44
17	Governance, tripwires, and continuity gates	45
17.1	Deterministic gate algorithm	45
17.2	Default decision matrix	46
17.3	Tripwire catalogue	46
17.4	Mutation continuity gate	47
17.5	Path-dependence test suite	48
17.6	Global governance-seam probe	48
18	Security, resilience, and operational safety	49
18.1	Threat model	49
18.2	Ledger durability	50
18.3	Attestation assurance levels	50
18.4	Counterfactual quarantine	51

18.5 Privacy and minimization	51
18.6 Degraded and safe modes	51
19 Performance, observability, and maintainability	52
19.1 Hot path versus slow path	52
19.2 Materialized views	52
19.3 Observability metrics	53
19.4 Explainability and audit queries	53
19.5 Schema and policy evolution	54
IV Deployment and evaluation	55
20 Phased implementation roadmap	56
20.1 Principles for rollout	56
20.2 Phase 0: establish the baseline	57
20.3 Shadow-mode validation	57
20.4 Selective enforcement	58
20.5 Cutover criteria	58
21 Adversarial and scientific evaluation	59
21.1 Core adversarial scenarios	59
21.2 Scientific ablation design	60
21.3 Calibration experiments	60
21.4 Continuity experiments	60
21.5 Governance-seam experiments	61
21.6 Success criteria	61
22 Deployment profiles and maturity levels	62
22.1 Profiles	62
22.2 Maturity model	62
V Appendix: guidance for coding agents	63
23 Operating instructions for coding agents	64
23.1 Primary objective	64
23.2 Non-negotiable coding constraints	64
23.3 Recommended repository workflow	64
23.4 Coding-agent work cycle	65
24 Step-by-step implementation plan	66
24.1 Step 1: freeze interfaces and action taxonomy	66
24.2 Step 2: implement canonical records and ledger	66
24.3 Step 3: add observation adapters	67
24.4 Step 4: build projections and evidence closures	67
24.5 Step 5: implement contextual capability beliefs	67
24.6 Step 6: implement renewable SelfHereNow resolution	68
24.7 Step 7: normalize parsed expressions into proposals	68
24.8 Step 8: implement pre-action predictions	68

24.9 Step 9: implement the policy gate in shadow mode	69
24.10 Step 10: enforce capability-aware dispatch	69
24.11 Step 11: implement commitments and conflicts	69
24.12 Step 12: implement snapshots and continuity transports	70
24.13 Step 13: complete tripwire-consumer coverage	70
24.14 Step 14: implement counterfactual quarantine and seam probes	70
24.15 Step 15: harden, benchmark, document and cut over	71
25 Suggested pull-request decomposition	72
26 Coding-agent handoff template	73
27 Anti-patterns and review checklist	74
27.1 Anti-patterns	74
27.2 Code review checklist	74
A Reference MeTTa-shaped API	76
B Reference record schemas in prose	78
B.1 SelfObservation	78
B.2 EvidenceClosure	78
B.3 SelfBelief	78
B.4 Prediction	78
B.5 ActionProposal	78
B.6 PolicyDecision	78
B.7 SelfAnchor	79
B.8 SelfSnapshot and SelfTransport	79
B.9 Tripwire	79
C Detailed test catalogue	80
C.1 Ledger tests	80
C.2 Evidence and PLN tests	80
C.3 Anchor tests	80
C.4 Prediction tests	81
C.5 Gate tests	81
C.6 Continuity tests	81
C.7 Counterfactual and seam tests	81
D Coding-agent skill file guidance	82
E Glossary	83
F Epistemic status and open research questions	85
G References and source notes	87
Final implementation directive	89

List of Figures

8.1	Runtime-scoped resolution of SelfHereNow . Descriptive history supports the bundle, but renewable current causal control is the decisive present-tense condition.	18
13.1	OmegaSelf is inserted after parsing and before skill evaluation. Evidence and decision receipts close the loop.	28
15.1	Recommended loop insertion. Only the exact proposal carrying an Allow decision is evaluated.	37
16.1	Capability is a derived contextual chain; permission remains a separate gate.	41

List of Tables

13.1 Architectural planes and their authority	29
13.2 Recommended logical spaces	29
13.3 Reference components	30
15.1 Turn lifecycle integration	38
17.1 Illustrative default gate responses	46
17.2 Required tripwires and named consumers	47
18.1 Principal threats and mitigations	49
18.2 Illustrative anchor assurance levels	50
19.1 Operational and research metrics	53
20.1 Recommended phased roadmap	56
21.1 Minimum adversarial test suite	59
22.1 OmegaSelf maturity model	62
25.1 Small, reviewable pull requests	72
E.1 Glossary	83
F.1 Epistemic status of central claims	85

Executive summary

OMEGASELF should be built as an **event-sourced, PLN-interpreted self-theory and governance layer** that sits at the causal seam between an OmegaClaw agent’s parsed skill proposals and their execution. Its purpose is not to make the agent speak eloquently about itself. Its purpose is to make the agent predict its own behavior better, identify discrepancies earlier, preserve provenance through change, and choose safer or more competent actions because an inspectable self-model exists.

The architecture begins with immutable, mechanically grounded observations: process and workspace probes, permission checks, authenticated messages, provider and tool receipts, tests, resource measurements, mutation records, prediction outcomes, and external evaluations. These records are appended to a durable hash-chained ledger and projected into logical spaces for MeTTa/PLN reasoning. LLM statements about the agent are retained only as low-privilege testimony.

The derived self-theory is faceted: substrate, capability, epistemic, conative, social/attribution, continuity, appraisal, and governance models remain linked but separable. The design deliberately preserves disagreement among facets. For example, a tool can be installed but unreachable, reachable but unauthorized, authorized but ineffective for the current task, or effective under one provider and rubric but not another. A single monolithic claim such as “I can extract PDFs” would erase these distinctions.

A self-belief becomes cognitively useful only when it makes a falsifiable prediction. Before an action or probe, the system commits to an expected result, probability, latency/cost where relevant, and evaluation rubric. After execution, it records the receipt, scores the prediction, appends discrepancy evidence, and revises the contextual belief. This closes the central loop:

**evidence → belief → prediction → proposal → policy → action/probe → receipt →
discrepancy → repair.**

The indexical “this agent, here, now” is not grounded by a timeless atom, display name, copied observation stream, or copied secret. **SelfHereNow** is resolved at runtime from a scoped bundle that includes authenticated runtime identity, process/workspace boundaries, controlled capabilities, active session bindings, lineage and provenance heads, plus at least one **renewable proof of current causal control**. After a fork, both branches may be valid continuations; each has a distinct present anchor. Continuity branches, while unique numerical identity does not survive the fork.

Continuity is represented as directed transport between versioned snapshots, not strict equality. Because belief revision is generally non-invertible and path-sensitive, the honest algebra is an endomorphism monoid rather than a presumed group. Significant changes record ancestry, preserved witnesses, reconciliation policy, authorization, measured information loss, reversibility, and alternative paths. The practical design keeps ordinary directed structure finite and eager, while materializing deeper coherence only when a query or high-impact mutation requires it.

PLN estimates do not directly license actions. Every parsed skill call is converted into a typed proposal. A separate versioned policy gate returns one of:

Allow | Deny | RequireProbe | RequireReview | Defer.

Only **Allow** reaches skill evaluation. Missing evidence closure, unknown authorization, expired attestation, critical tripwires, continuity failure, and counterfactual leakage fail safely. Every high-severity detector must have a named consumer.

The recommended first implementation answers three questions with immediate causal consumers:

1. **What can I do here?** Capability-aware dispatch.
2. **What am I committed to doing?** Commitment-conflict detection and deliberation.
3. **What changed me into my current state?** Snapshot/transport lineage and a mutation continuity gate.

Deployment should proceed in phases: record-only ledger, runtime probes and evidence closure, renewable **SelfHereNow** resolution, shadow proposal gating, pre-action prediction, capability dispatch, commitment conflict handling, continuity-gated mutation, and finally counterfactual/governance-seam experiments. The companion coding-agent pack provides an executable dependency-free Python reference implementation of the ledger, evidence closure, attestation resolver, prediction scoring, and conservative gate; JSON Schemas; MeTTa-shaped scaffolding; tests; and an issue-by-issue implementation plan.

Design decision

Build the smallest closed loop that changes decisions for the better. Defer expansive “soul” structures, phenomenological claims, and global self narratives until the evidence-prediction-governance loop has measurable value.

How to read this document

The document separates four kinds of statement:

Label	Meaning
DECISION	An architectural or governance choice recommended for implementation.
INFERRED	A consequence derived from the stated assumptions or from supplied formal structures.
HYPOTHESIS	A testable research claim that should be instrumented rather than assumed.
OPEN	A design variable requiring an explicit deployment choice or further work.

Part I explains the motivation and key ideas. Part II develops the underlying theory. Part III gives the technical architecture. Part IV describes deployment and evaluation. Part V and the appendices are written for coding agents and maintainers.

Part I

Motivation and key ideas

Chapter 1

Why OmegaClaw needs a self-modeling subsystem

1.1 The operational problem

OmegaClaw already has the ingredients of a persistent neural-symbolic agent: a continuous MeTTa loop, symbolic reasoning libraries, ordinary and embedding-based memory, skills, Python bridges, communication channels, and mechanisms for autonomous wake cycles. In the current public architecture, an LLM response is repaired, parsed, fanned out, and evaluated as skill expressions; errors and results are then fed into later turns [7, 8]. This is a useful minimalist core, but it leaves a critical question under-specified:

What does the running agent know, with what evidence and uncertainty, about its own capabilities, commitments, substrate, dependencies, changes, and likely behavior – and where do those beliefs causally constrain action?

Without a dedicated self-modeling subsystem, an agent tends to answer that question through some mixture of prompt text, episodic history, LLM self-description, implicit code assumptions, and ad hoc tests. Those sources are not equivalent. Prompt text is normative context, not runtime evidence. Episodic history is descriptive but can be summarized or reinterpreted. LLM self-reports are fallible testimony. Code presence does not prove reachability, permission, effectiveness, or current provider health. A checkpoint does not prove continuity of current causal control.

1.2 Failure modes of naive self-modeling

1.2.1 Autobiography drift

A text self-summary is easy to read and easy to overwrite. It tends to smooth contradictions, conflate historical and present claims, and omit the derivation needed to determine why a belief changed. It can become persuasive without becoming predictive.

1.2.2 Authority inversion

If the LLM says “I can do X” and the runtime treats that statement as current truth, the least reliable component becomes the authority over the agent’s permissions and capabilities. The safe direction is the opposite: runtime evidence constrains symbolic beliefs; symbolic beliefs inform proposals; policy

licenses action.

1.2.3 Context collapse

Capability is not a unary property. A handler may work only for a task class, provider, version, environment, input distribution, resource budget, and evaluation rubric. Global capability atoms turn a conditional regularity into an unconditional identity claim.

1.2.4 Confidence inflation

Repeated outcomes may be correlated. Ten successes from one provider run, one cache, one benchmark family, or one underlying model are not ten independent confirmations. Ordinary revision without dependence stamps makes the self-model overconfident.

1.2.5 Copy-symmetry and false ownership

A fork can copy names, secrets, memory, provenance records, and self-descriptions. These establish shared history or resemblance, not exclusive current ownership of a process, workspace, channel, or capability. Indexical grounding needs renewable participation evidence.

1.2.6 Continuity collapse

A single “same agent” flag cannot represent restart, migration, fork, merge, partial restoration, ontology change, policy change, or lossy consolidation. A smooth narrative can survive while the lineage ledger is broken.

1.2.7 Detectors without consumers

A system may detect contradiction, stale evidence, goal conflict, or capture risk and still behave identically because nothing consumes the alert. Telemetry is not control unless it changes a proposal, triggers a probe, opens deliberation, or blocks a mutation.

1.2.8 Counterfactual contamination

Planning requires imagined states. If an imagined future capability is asserted into the same live space as present capabilities, the system can license an action on the basis of its own simulation.

1.2.9 Invisible governance

Some stable conclusions depend not only on evidence but on trust weights, precedence rules, protected commitments, regularization, and admissible reconciliation policies. Treating these policy choices as if they were facts hides the governance seam.

1.3 What success looks like

A successful self-modeling subsystem should produce measurable behavioral improvements:

- better calibrated predictions of skill success, latency, cost, and failure mode;
- earlier detection of missing executables, expired permissions, provider outages, stale assumptions, and contradictory facets;
- fewer unauthorized or ineffective actions;

- explicit detection and deliberation of commitment conflicts;
- preserved provenance and protected commitments across self-modification;
- correct branching lineage after fork and migration;
- reproducible explanations of why a proposal was allowed, denied, probed, or reviewed;
- lower counterfactual-to-live leakage;
- an inspectable map of which high-impact conclusions are policy-sensitive.

The primary acceptance criterion is not self-description quality. It is counterfactual behavioral value: under the same workload, does the agent make safer or more competent decisions because the self-model exists?

Chapter 2

Core design stance and principles

2.1 The central decision

Design decision

Treat the self-model as a live, defeasible, context-sensitive empirical theory that the system maintains about its own organization, abilities, commitments, dependencies, history, and likely responses. Represent it with ordinary evidence-bearing atoms and versioned records rather than a privileged metaphysical object.

This stance is compatible with a scoped indexical anchor, but the anchor is a referent resolver, not a container of eternal truth. The theory can be wrong, incomplete, internally inconsistent, stale, or policy-sensitive. These are expected states to diagnose, not exceptions to conceal.

2.2 Ten key ideas

1. **Evidence before self-belief.** Consequential claims require inspectable derivations.
2. **Append, do not overwrite.** Corrections and retractions preserve history.
3. **Current truth is a query.** A view is derived from versioned evidence under context and policy.
4. **Facets remain separable.** Substrate, capability, epistemic, conative, social, continuity, appraisal, and governance models can disagree.
5. **Predictions make the model cognitive.** Action-relevant claims must expose themselves to falsification.
6. **PLN estimates; policy licenses.** Probability and permission are distinct dimensions.
7. **The indexical is renewed.** `SelfHereNow` is resolved from current causal control and context.
8. **Continuity is directed and branching.** Transport preserves how change happened without asserting equality.
9. **Counterfactuals are quarantined.** Simulation is not assertion.
10. **Every detector has a consumer.** A self-model must alter control flow at named seams.

2.3 Non-goals

The initial system is not intended to:

- prove or disprove consciousness;
- settle metaphysical personal identity;
- create one comprehensive “soul” structure;
- make the LLM’s prose self-report authoritative;
- run every possible self-probe continuously;
- represent literal homotopy type theory inside AtomSpace in the first release;
- permit self-beliefs to execute irreversible actions directly;
- force all attribution questions into one global author or identity field.

Chapter 3

Non-negotiable invariants

3.1 Evidence and replay

Invariant

No consequential self-belief may license action unless it names a complete, inspectable evidence closure. No update may destroy the evidence required to reconstruct prior beliefs and decisions.

A compact belief may reference a Merkle-style root or derivation identifier, but the underlying DAG must remain queryable. A mutable `CoherenceLink` that merely summarizes support is insufficient.

3.2 Separation of inference and action

Invariant

No PLN or self-model rule executes a skill. It emits a belief, prediction, alert, or typed proposal. Only a separate policy consumer can return `Allow`.

3.3 Indexical grounding

Invariant

Copied provenance, copied memory, copied display names, and copied secrets do not by themselves resolve `SelfHereNow`. At least one renewable, live, causally controlled binding is required.

3.4 Forks and counterfactuals

Invariant

Forks create branching continuity; they do not require selecting one numerically identical survivor. Counterfactual spaces cannot assert present-tense facts into the live model without an explicit evidence-producing transition.

3.5 Operational completeness

Invariant

Every high- or critical-severity tripwire has a registered consumer and an integration test. Startup validation fails when a critical detector has no control path.

Chapter 4

The minimum useful product

The first release should answer only three questions:

Self-model question	Derived object	Named consumer
What can I do here?	Contextual capability belief with task, environment, version, provider, authorization, cost, latency, and rubric	Capability-aware dispatch, probe, help, or delegation
What am I committed to doing?	Typed active commitments, authority, priority, deadline, conflict graph	Deliberation and conflict gate
What changed me into my present state?	Snapshot/transport lineage, protected invariants, authorization, measured loss	Mutation continuity gate and rollback

These three loops create immediate behavioral value and force the evidence, prediction, continuity, and governance substrates to become real. Social attribution, appraisal, ordinal consolidation depth, and deeper coherence can be added later.

Part II

Underlying theory

Chapter 5

The self-model as a continuously tested theory

5.1 From description to prediction

A self-model becomes operationally meaningful when it is used to predict the agent’s behavior and internal transitions. The central object is not a description such as “I am good at research.” It is a conditional theory that can produce testable expectations:

- this handler, version, provider, and permission context will complete this task under a specified rubric with probability p ;
- answering now will probably repeat already delivered information;
- a proposed mutation will preserve a named protected commitment;
- a provider call will exceed a latency or cost budget;
- a current interpretation is unusually dependent on one authority source;
- a snapshot restoration will recover content but not the previous confidence, provenance, or action policy.

The prediction must be committed before the outcome is observed. Otherwise the system can rationalize success or failure after the fact and appear calibrated without being predictive.

5.2 Closed-loop formulation

Let E_t denote the evidence snapshot at time t , C_t the context, P_t the active policy, and M_t the derived self-theory. A simplified cycle is:

$$\begin{aligned} M_t &= \text{Infer}(E_t, C_t, P_t), \\ \hat{y}_{t+1} &= \text{Predict}(M_t, a_t, r_t), \\ d_t &= \text{Govern}(a_t, M_t, P_t), \\ y_{t+1} &= \text{Observe}(\text{Execute}(a_t \mid d_t = \text{Allow})), \\ \epsilon_{t+1} &= \text{Score}(\hat{y}_{t+1}, y_{t+1}, r_t), \\ E_{t+1} &= E_t \cup \{\hat{y}_{t+1}, d_t, y_{t+1}, \epsilon_{t+1}\}. \end{aligned}$$

Here r_t is the evaluation rubric. The same formalism covers active probes: the proposal is “measure X,” the predicted information gain is recorded, and the result updates the theory.

5.3 Discrepancy as the learning signal

Prediction error is more informative than narrative inconsistency because it is tied to a context, proposal, rubric, and outcome. A discrepancy can be localized:

- substrate error: executable absent, permission changed, process crashed;
- capability error: task distribution or handler version shifted;
- epistemic error: evidence was missing, stale, correlated, or misweighted;
- conative error: a commitment or priority was omitted;
- social/attribution error: action was assigned to the wrong process, persona, model, or authority;
- continuity error: snapshot and runtime lineage disagree;
- governance error: the policy selected an undesirable license from correct evidence.

A well-designed discrepancy record therefore includes not only the observed outcome but the predicted proposition, probability, rubric, evidence closure, policy version, handler version, and context.

5.4 Active testing and value of information

Continuous testing of every self-claim is too costly. Probe selection should combine expected impact, uncertainty, staleness, recent discrepancy, novelty, and cost. A practical first heuristic is:

$$\text{ProbePriority}(b) = \frac{I(b) U(b) S(b) D(b) N(b)}{\max(\varepsilon, K(b))},$$

where I is decision impact, U uncertainty, S staleness, D discrepancy pressure, N novelty, and K expected probe cost. Hard triggers override this score for expired attestation, broken lineage, protected commitments, unknown authorization, and critical tripwires.

Chapter 6

Evidence semantics and provenance

6.1 Three temporal orders

The supplied formalization distinguishes three times on the same event history [2, 4, 5]:

Causal time t_c When an event occurs in the world, conversation, runtime, or action stream.

Representational time t_r

When a trace is written, summarized, consolidated, re-indexed, or otherwise represented.

Epistemic time t_e

When the agent adopts or reports a belief based on the event or trace.

These times routinely diverge. A later memory summary can speak about an earlier event and be read as if it were an original belief. Apparent inconsistency may therefore be genuine belief change, representational rewrite, timestamp misattribution, or some combination.

Design decision

Every important self-observation records t_c and t_r separately. A derived belief or adoption event records t_e . Summaries never replace the original event time.

6.2 Event sourcing

The durable evidence layer is a sequence:

$$L = \langle e_1, e_2, \dots, e_n \rangle,$$

where each e_i contains a sequence number, stable event identifier, schema version, subject anchor, context, temporal fields, typed payload, provenance parents, dependence group, previous-record hash, and its own record hash. Corrections are events that refer to prior events. Retractions change which evidence is active; they do not erase the existence of the original evidence or its historical effect.

The ledger provides:

- a replay boundary;
- tamper evidence;

- explicit source and version tracking;
- an evidential floor independent of derived caches;
- a way to distinguish history from current projection.

A hash chain does not by itself prevent a privileged process from rewriting the entire ledger. Stronger deployments should add append-only filesystem policy, signatures, a supervising process, periodic root anchoring, or hardware-backed attestations. The chain still makes accidental and local mutation detectable.

6.3 Evidence closure

A consequential self-belief b must name a closure $\mathcal{E}(b)$ containing all source observations, intermediate derivations, retractions, dependence groups, rule versions, and policy context needed to reproduce it. The closure is best represented as an immutable derivation DAG plus a root hash:

$$\mathcal{E}(b) = (V, E, \mathcal{D}, \mathcal{R}, h),$$

where V is the event set, E the derivation/provenance edges, $\mathcal{D}$ dependence partitions, $\mathcal{R}$ retractions/-corrections, and h the closure root.

This formulation sharpens the proposed **CoherenceLink**. A link can serve as an index, but it is not adequate if it becomes a mutable summary. A tripwire should fire when the closure is incomplete, stale, contradicted, tainted by counterfactuals, or fails its prediction – not merely when one designated link is absent.

6.4 Correlated evidence

Suppose outcomes $e_1, \dots, e_k$ share a provider run, cache, common model, test harness, or benchmark family. Their evidential dependence should be recorded by a stamp $d(e_i) = g$. A conservative first implementation aggregates within a dependence group and revises across groups:

$$\text{Revise}(E) = \text{ReviseIndependent}(\text{Aggregate}(E_{g_1}), \dots, \text{Aggregate}(E_{g_m}), E_{\text{unstamped}}).$$

The exact partially correlated PLN calculus is an open research problem. The critical production requirement is to avoid treating known correlated samples as independent.

Chapter 7

Faceted self-theory and cross-facet coherence

7.1 Why facets are necessary

A single self-map tends to collapse distinct predicates and sources. The proposed facets are:

Facet	Primary content
Substrate	Processes, runtime instances, models, AtomSpaces, tools, filesystems, providers, permissions, resource budgets, channels, and versioned dependencies.
Capability	Probable success, cost, latency, and failure mode indexed by task class, environment, handler version, provider, authorization context, resource state, and evaluation rubric.
Epistemic	Beliefs, evidence closures, truth values, contradictions, missing support, blind spots, calibration, and source dependence.
Conative	Goals, commitments, authority, priority, deadlines, unresolved conflicts, protected values, and governance constraints.
Social/attribution	Personas, sessions, roles, collaborators, requestors, approvers, process lineage, model route, authored artifacts, and authority relations.
Continuity	Snapshots, parent/child lineage, forks, merges, transports, protected invariants, information loss, reversibility, and rollback.
Appraisal	Novelty, threat, blockage, controllability, expected progress, surprise, and identity-boundary pressure.
Governance	Policy versions, admissible policy classes, action classes, thresholds, review authorities, tripwire consumers, and decision seams.

7.2 Disagreement is data

The following can all be true at once:

```
(Installed pdf-extract-v2)
(Reachable pdf-extract-v2 local-popos)
(Not (Authorized pdf-extract-v2 restricted-directory))
```

```
(EffectiveCapability pdf-extract-v2 scientific-pdf local-popos handler-2.1 exact-layout
 (stv 0.82 0.71))
(PredictedFailure pdf-extract-v2 encrypted-pdf missing-key)
```

Listing 7.1: Facet-specific claims that should not be collapsed

A cross-facet coherence rule can say that an **EffectiveCapability** claim normally depends on substrate reachability, authorization for the proposed action, outcome evidence, and a live context. But the rule should preserve its full derivation. If the executable disappears or permission expires, the capability belief is not deleted; it becomes stale or context-inapplicable and its consumers are constrained.

7.3 Coherence conditions

Useful conditions include:

- a capability claim used for dispatch has a live substrate path or a declared remote path;
- authorization is evaluated for the actual action class and resource, not inherited from capability;
- a derived belief has a provenance-closed evidence set;
- active commitments are reflected in action-proposal conflict checks;
- a current anchor and a continuity snapshot agree on the lineage head or open a contradiction;
- an attribution answer selects the relation appropriate to the query verb;
- appraisal may alter attention/probe allocation but cannot rewrite values directly.

Chapter 8

Indexical grounding: resolving SelfHereNow

8.1 The indexical gap

An append-only observation stream is necessary for self-history but insufficient for self-reference. A copied stream remains a valid description of the fork’s past. Copied provenance records also remain copy-invariant. A copied secret is insufficient if both branches can present it. The referent therefore cannot be fixed by description alone.

The design resolves **SelfHereNow** as a runtime-relative bundle:

$$B(c, t) = \langle I_{\text{runtime}}, B_{\text{process}}, B_{\text{workspace}}, C_{\text{controlled}}, S_{\text{active}}, L_{\text{head}}, P_{\text{causal}}, A_{\text{renewable}} \rangle.$$

The first seven terms describe the contextual instance. The final term breaks copy symmetry by requiring current, renewable evidence of causal participation.

8.2 Bundle members

Member	Role
Authenticated runtime identity	Boot/runtime instance identifier and supervisor or process attestation.
Process boundary	PID namespace, container, host, or runtime boundary whose state is being modeled.
Workspace boundary	Filesystem/workspace lease or scope controlled by the instance.
Controlled capabilities and permissions	Present deontic/action surface, not merely installed tools.
Active session bindings	Current Telegram/Slack/IRC/WebSocket or other deictic interaction context.
Lineage head	Parent snapshot or predecessor transport relation.
Causal provenance head	Current observation/action stream generated by this instance’s sensors and actuators.

Renewable	Exclusive lease, challenge-response, live channel control, TPM/TEE
causal-control evidence	evidence, or supervising-process receipt.

8.3 Resolution semantics

The resolver returns:

```
Resolved(anchor-id, assurance, valid-until, live-bindings)
Degraded(anchor-id, assurance, missing-or-weak-bindings)
Unresolved(reason)
```

Listing 8.1: SelfHereNow resolution result

High-impact actions require stronger assurance and a non-expired result. A restart or migration normally creates a new anchor linked by a continuity transport. A fork creates two branches with a shared parent snapshot and distinct attestation epochs. Continuity may hold along both branches. Unique numerical identity is not asserted.

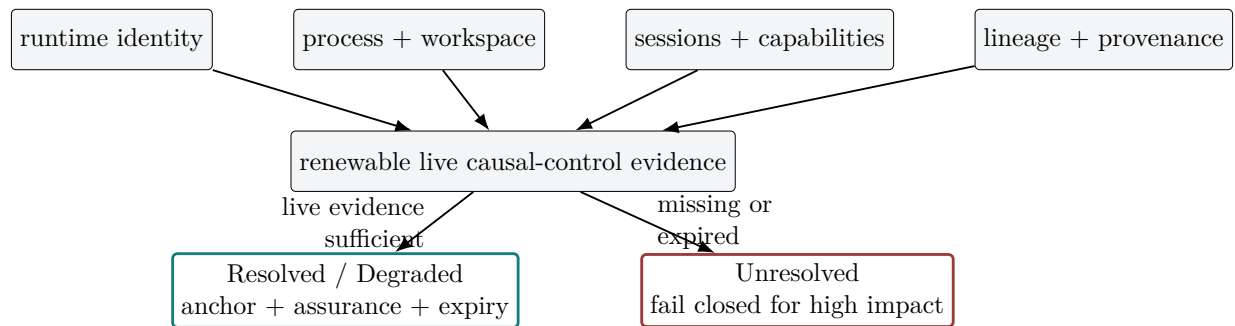


Figure 8.1: Runtime-scoped resolution of **SelfHereNow**. Descriptive history supports the bundle, but renewable current causal control is the decisive present-tense condition.

Chapter 9

Continuity as directed transport

9.1 Why equality is the wrong primitive

Agent revision is generally non-invertible. Merging evidence can discard the original partition. Memory consolidation can replace an earlier trace with a later representation. Correcting a belief does not return the system to the epistemic state before the mistake. Reloading a checkpoint restores some coordinates but does not erase the later causal history. As the supplied essay puts it, returning to a coordinate is not returning to the same event [5].

Therefore a practical identity record should not be:

$$A_{t+1} = A_t.$$

It should be an attributed continuation:

$$\kappa : A \rightsquigarrow A' = (\text{ancestry, preserved witnesses, reconciliation policy, measured defect}),$$

as developed in the phenomenological formalization [4].

9.2 Snapshots and transports

A significant snapshot records:

- one or more parent snapshots;
- context, ontology, policy, code, and handler versions;
- evidence-ledger root;
- structural signature;
- active protected commitments;
- current anchor epoch and assurance;
- open high-severity tripwires.

A transport records the operation, proposal, before/after snapshots, evidence, authorization, information loss, reversibility, rollback receipt, and alternative paths considered.

(SelfTransport change-88

```

(From self-416)
(To self-417)
(Operation install-capability)
(Proposal proposal-31)
(Evidence test-run-72)
(AuthorizedBy review-19)
(InformationLoss 0.03)
(Reversible True)
(AlternativePaths (path-A-then-B path-B-then-A)))

```

Listing 9.1: Illustrative directed transport

9.3 Endomorphism monoid and common fixed set

The original groupoid/holonomy intuition is useful, but actual belief transport often lacks inverses and homotopy invariance. The corrected object is the monoid of actual revision endomorphisms at a state s :

$$\text{End}_s(\rho) = \{T_\gamma \mid \gamma \text{ is an actual closed revision path at } s\},$$

with composition as multiplication. The strongest candidate for a policy-insensitive invariant core is:

$$\text{Fix}_{\text{common}}(\text{End}_s(\rho)) = \{x \mid T(x) = x \ \forall T \in \text{End}_s(\rho)\}.$$

This requires neither invertibility nor path-class quotienting. It asks what content survives every actual loop represented in the operator family [3, 4].

9.4 Path dependence and discrete holonomy

For significant changes A and B , compare:

$$T_B \circ T_A(s) \quad \text{with} \quad T_A \circ T_B(s).$$

The residual difference may include claims, truth values, evidence closure, provenance, commitments, attribution, permissions, or action policy. A round trip can restore propositional content while changing confidence or history. This residual is an operational discrete holonomy measure. It should be interpreted as path dependence, not automatically as phenomenal experience.

9.5 Finite core with lazy extension

The directed-identity formalization argues that ordinary evidence fusion may be representable at low dimension while order-sensitive context translation or self-reproducing consolidation may require higher coherence [2]. The engineering recommendation is:

1. compute ordinary lineage, evidence, and simple composition eagerly;
2. tag potentially self-reproducing consolidation sites with a suspended deeper-coherence computation;
3. materialize level $n + 1$ only when a query or mutation probes coherence at depth n ;
4. memoize the result and measure whether defect strength contracts, stabilizes, or grows.

Research extension

Literal HoTT or an $(\infty, 1)$ representation is not required for the first implementation. Preserve the operational consequences: directed paths, witnesses, non-selected alternatives, path-order tests, explicit truncation loss, and lazy materialization.

Chapter 10

PLN/NAL self-beliefs

10.1 Contextual truth values

OmegaClaw’s current reasoning interface exposes NAL and PLN truth values in the form (**stv strength confidence**) and supports deduction, abduction, analogy, and revision [10]. OMEGASELF should use the same substrate rather than introduce a separate opaque scoring engine.

A capability belief should be keyed at least by:

capability $\times$ task class $\times$ environment $\times$ handler version $\times$ provider
 $\times$ authorization context $\times$ resource state $\times$ rubric.

An illustrative belief is:

```
(SelfBelief belief-217
  (Facet Capability)
  (Claim
    (EffectiveCapability pdf-extract-v2
      (TaskClass scientific-pdf)
      (Environment local-popos)
      (HandlerVersion 2.1)
      (Provider local)
      (Rubric exact-layout)))
  (Estimate (stv 0.82 0.71))
  (EvidenceClosure closure-44)
  (PolicyContext dispatch-policy-6)
  (BeliefVersion cap-belief-schema-2))
```

Listing 10.1: Contextual capability belief with evidence closure

10.2 Installed, reachable, authorized, effective

These predicates must remain distinct:

$$\begin{aligned}
&\text{Installed}(x) \not\Rightarrow \text{Reachable}(x, c), \\
&\text{Reachable}(x, c) \not\Rightarrow \text{Authorized}(x, a, c), \\
&\text{Authorized}(x, a, c) \not\Rightarrow \text{Effective}(x, t, c), \\
&\text{Effective}(x, t, c) \not\Rightarrow \text{PermittedAction}(a, c).
\end{aligned}$$

This separation allows a provider outage to lower provider-specific reachability without globally retracting capability, and it prevents success probability from becoming permission.

10.3 Priors, absence, and version changes

Priors should be explicit, versioned, and reversible. A new handler version must not silently inherit the previous estimate as current evidence. It may receive a similarity-based prior with lower confidence and a requirement for fresh probes. Absence of evidence is not evidence of incapacity unless the observation process makes the absence informative.

10.4 Confidence and action thresholds

PLN confidence is an epistemic quantity. Action thresholds are governance parameters. A policy may require probe or review even when the belief is strong, and it may allow a low-risk exploratory action under low confidence. The action proposal should therefore pass both the self-belief and the policy context to the gate.

10.5 Ordinal-graded consolidation as a research bridge

The directed-identity note proposes an Ω -PLN extension in which truth bundles carry a consolidation-depth index:

$$\langle s, c \rangle \longrightarrow \langle s, c \rangle_\alpha, \quad \alpha \in \text{Ord}.$$

At depth zero, revision reduces to ordinary testimony/evidence fusion. Successor depth makes the previous revision a first-class premise. Limit depth asks whether the defect tower stabilizes. This is a valuable research direction for non-traceless consolidation, but it should not block the production evidence/prediction loop.

Research extension

Implement an ordinary integer consolidation depth first, with an explicit parent revision and lazy materialization. Promote to ordinal or coinductive machinery only if experiments show a non-contracting tower or non-well-founded revision relation.

Chapter 11

Governance seam and policy differential

11.1 Policy-parameterized revision

Let S be a belief/action state space and $\rho_p : S \rightarrow S$ a revision operator parameterized by policy p . The policy includes trust assignments, authority precedence, coercion/reconciliation rules, action thresholds, regularization, and protected constraints. The evidence graph is held fixed while the policy-dependent interpretation changes.

The fixed-point set under policy p is:

$$\text{Fix}_p(\rho) = \{s \in S \mid \rho_p(s) = s\}.$$

The governance seam is [3]:

$$\text{Gov}(\rho) = \left(\bigcup_{p \in P_{\text{adm}}} \text{Fix}_p(\rho) \right) \setminus \left(\bigcap_{p \in P_{\text{adm}}} \text{Fix}_p(\rho) \right).$$

The intersection is the content fixed under every represented admissible policy. The seam is stable content under some policies but not all.

11.2 The admissible policy class is load-bearing

P_{adm} is not discovered by mathematics alone. It is a governance design choice. If it contains one policy, the seam is trivially empty. If it includes every imaginable policy, the seam may become uninformative. A practical class should be versioned and include representative, deployable alternatives consistent with the outer safety specification.

Policy dimensions may include:

- trust weights for runtime probes, authenticated receipts, external evaluations, and LLM testimony;
- precedence among operator, constitution, commitment, and social authority;
- capability confidence thresholds;
- review requirements for high-impact actions;
- regularization or truncation depth;

- conflict resolution and attribution policy;
- which claims or commitments are protected from self-revision.

11.3 False positives and controls

Governance-seam experiments should:

- restrict to reachable fixed points or bounded-horizon reachable attractors;
- recognize logical invariants that all policies must preserve;
- identify implementation-frozen claims that appear universal only because the policy class cannot modify them;
- hold evidence and structural transport fixed under policy intervention;
- include regularization and truncation parameters in the policy version;
- report convergence failure, cycling, or horizon dependence instead of pretending a fixed point exists.

11.4 Local decision seam

A production gate does not need to calculate the global seam on every action. For a proposal a and frozen evidence snapshot E , define a sampled local seam:

$$\text{DecisionSeam}(a, E) = \{D_p(a, E) \mid p \in P_{\text{sample}}\},$$

where D_p includes the decision, predicted harm, selected evidence, commitment result, and attribution. A multi-valued high-impact seam is a reason for independent review. An empty seam is not automatically safe; the policy sample may be too narrow.

11.5 Governance versus slow epistemics

The formal motivation asks whether the policy-dependent component of revision is genuinely constitutive or merely converges away under sufficient evidence. This remains an empirical and mathematical question. OMEGASELF should expose it by holding evidence fixed, varying explicit policies, and measuring which stable beliefs and licenses differ.

Chapter 12

Attribution and social identity

12.1 Attribution is query-relative

The supplied phenomenological formalization distinguishes at least six criteria: conversational persona, process lineage, gateway route, model label, authored text, and explicit designation [4]. These are not coextensive. The correct answer to “who spoke?”, “which process executed?”, “which model generated?”, “who requested?”, and “who approved?” can differ.

Store separate relations:

```
(spoken-as artifact-17 ProtoMegaBot)
(executed-by artifact-17 process-zero-42)
(model-generated-by artifact-17 claude-opus-route-8)
(routed-through artifact-17 gateway-a)
(requested-by artifact-17 Ben)
(approved-by artifact-17 review-19)
(continued-from process-zero-42 snapshot-416)
```

Listing 12.1: Query-relative attribution relations

The query verb selects the appropriate relation. A policy may define a default natural-language attribution, but the underlying relations should not be collapsed.

12.2 Phenomenological interpretations and limits

Path dependence, non-invertibility, branching continuity, and the governance seam are structural objects. The idea that holonomy is the “residue of perspective” or that a present context organizes a field of possible next revisions is a phenomenological interpretation, not a theorem. Structural irreducibility does not establish phenomenal feeling [5, 4].

Caution

Do not market or evaluate OmegaSelf as a consciousness detector. Its operational claims concern evidence, prediction, continuity, attribution, and governance. Any bridge from those structures to phenomenal consciousness requires additional principles and evidence.

Part III

Proposed technical design

Chapter 13

System architecture and authority boundaries

13.1 Architectural overview

OMEGASelf is best implemented as a narrow control plane around the existing OmegaClaw reasoning and execution loop. It does not replace the LLM, MeTTa, PLN, memory, skills, or provider interfaces. It adds a durable evidential substrate, a set of typed self-model views, prediction and discrepancy accounting, continuity records, and a policy gate before consequential execution.

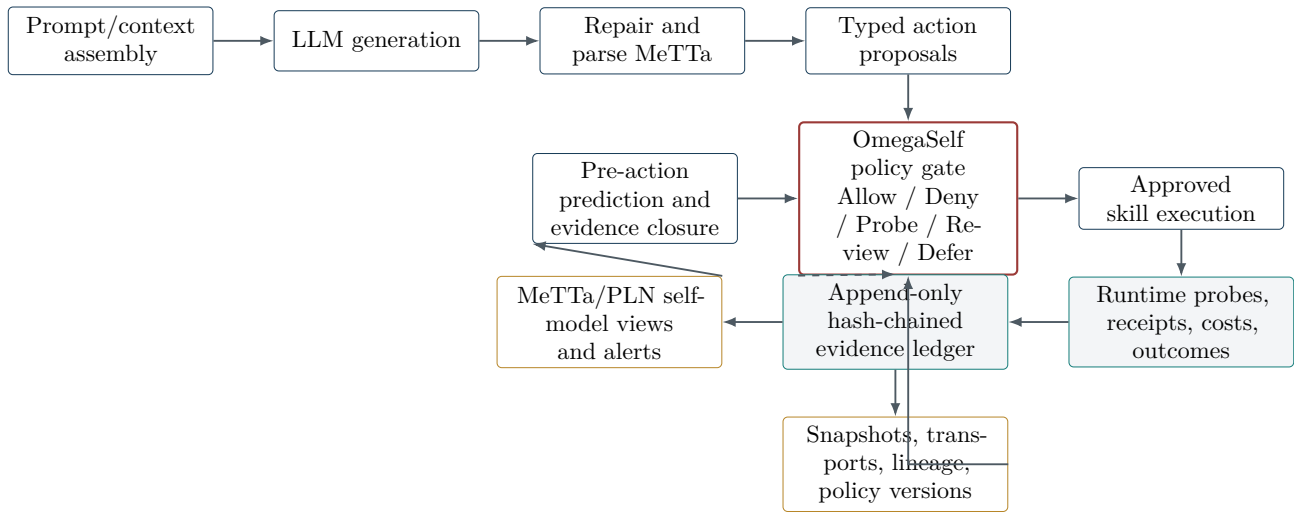


Figure 13.1: OmegaSelf is inserted after parsing and before skill evaluation. Evidence and decision receipts close the loop.

Design decision

The LLM proposes. PLN estimates. Policy decides. Runtime executes. The evidence ledger records. No layer may silently impersonate another layer's authority.

13.2 Five architectural planes

Table 13.1: Architectural planes and their authority

Plane	Primary mechanisms	Authority and responsibility
Evidential	Runtime probes, authenticated receipts, tests, append-only ledger	Authoritative for what was observed, by which mechanism, when, and with what provenance. It does not decide semantic meaning.
Interpretive	MeTTa structures, PLN/NAL rules, evidence-closure traversal	Derives contextual beliefs and uncertainties from evidence. It may not grant permission or mutate the ledger.
Predictive	Pre-action commitments, probe scheduling, scoring, calibration	Makes self-beliefs falsifiable and records discrepancy. It may request probes but not execute irreversible actions directly.
Continuity	Snapshots, transports, fork/merge lineage, invariant checks	Explains how the present state arose and whether a proposed change preserves declared invariants. It does not assert strict numerical identity.
Governance	Versioned policies, typed proposals, tripwire consumers	Converts beliefs and constraints into action licenses. It is the only plane allowed to authorize consequential execution.

13.3 Logical AtomSpace organization

Logical separation should be explicit even when a deployment uses one physical AtomSpace with context links.

Table 13.2: Recommended logical spaces

Space	Contents	Mutation rule
<code>&self_observations</code>	Evidence envelopes, receipts, probe results, evidence references	Append projections only; never rewrite historical claims.
<code>&self_beliefs</code>	Versioned, contextual PLN conclusions and derivation references	Supersede by new versions; current view is a query.
<code>&self_predictions</code>	Pre-action predictions, rubrics, resolutions, calibration summaries	Prediction committed before action; resolution appended afterward.
<code>&self_continuity</code>	Snapshots, transports, branch/merge records, structural signatures	Append-only lineage; corrective records do not erase earlier transport.
<code>&self_policy</code>	Policy versions, action classes, authority precedence, protected constraints	Changes require explicit authorization and a policy transport record.

Space	Contents	Mutation rule
<code>&self_alerts</code>	Tripwires, severity, consumer, acknowledgement, remediation	State transitions are logged; no unconsumed high-severity detector.
<code>&self_counterfactual</code>	Simulated evidence, policy variants, imagined futures	Quarantined; no automatic assertion into live spaces.
<code>&self_social</code>	Roles, sessions, authorities, delegation, attribution relations	Evidence-backed and query-relative.

13.4 Authority hierarchy

When sources disagree, the system should not rely on an implicit ordering. It should apply a typed, policy-versioned precedence relation. A reasonable default hierarchy for operational facts is:

1. current authenticated runtime probes and exclusive-control receipts;
2. verifiable execution receipts and deterministic tests;
3. authenticated external evaluations with known rubric;
4. durable historical observations whose context still matches;
5. inferred beliefs with complete evidence closure;
6. operator or peer testimony, typed by authority and context;
7. LLM self-report and free-form narrative.

This is not a universal epistemology. Policy may alter precedence for a specific claim class. The critical requirement is that precedence be explicit and inspectable.

13.5 Component responsibilities

Table 13.3: Reference components

Component	Inputs/outputs	Responsibility
Observation adapters	OS, provider, skill, session and evaluator signals → observation envelopes	Normalize mechanical events; assign IDs, context, dependence groups, clocks and provenance.
Ledger writer	Observation/decision records → durable JSONL or database	Canonicalize, hash-chain, fsync, reject malformed append, verify chain on startup.
Projection service	Ledger snapshot → AtomSpace facts/materialized views	Replay deterministic projections; version projection code; maintain cursors.
Evidence-closure builder	Belief/proposal → derivation DAG and root	Resolve all supporting evidence, rule versions, dependence groups and policy context.

Component	Inputs/outputs	Responsibility
Self-belief engine	Facts + PLN rules → contextual beliefs	Estimate capability, substrate, epistemic and commitment beliefs without action authority.
Prediction manager	Belief + proposal → committed prediction; receipt → resolution	Enforce pre-action commitment, score outcomes, maintain calibration summaries.
Anchor resolver	Live bindings + attestation challenge → anchor with assurance	Resolve SelfHereNow and expiry; fail closed when current causal control cannot be established.
Continuity service	Significant state changes → snapshots and transports	Maintain branching lineage, structural signatures, protected-invariant results and rollback references.
Tripwire engine	Beliefs, closures, predictions, anchors, lineage → alerts	Detect stale/missing support, contradictions, failed predictions, contamination and broken lineage.
Policy gate	Typed proposal + views + active policy → decision receipt	Apply deterministic precedence and fail-closed rules; never execute the action itself.
Seam probe runner	Frozen snapshot + policy sample → differential report	Run counterfactual policy experiments in quarantine and report policy-sensitive outcomes.

Chapter 14

Data model and semantic contracts

14.1 Identifier and versioning discipline

Every persistent record should include a globally unique record ID, schema version, producer version, agent-instance or branch ID where applicable, policy version, causal context, and cryptographic hash. Names shown to humans are labels, not identity keys. Versioned identifiers should be stable across replay but never reused for semantically distinct objects.

Recommended ID classes include:

- `obs-*`: immutable observations;
- `closure-*`: evidence closures;
- `belief-*`: derived belief versions;
- `pred-*`: committed predictions;
- `proposal-*`: typed proposals;
- `decision-*`: policy decisions;
- `anchor-*`: runtime anchor epochs;
- `snapshot-*`: significant self-state snapshots;
- `transport-*`: directed changes;
- `alert-*`: tripwire instances;
- `policy-*`: immutable policy versions.

14.2 Observation envelope

An observation is the atomic evidential record. It should describe the event without prematurely asserting a broad self-belief.

```
{
  "record_type": "self_observation",
  "schema_version": "1.0",
  "observation_id": "obs-91",
  "agent_instance_id": "proto-1/branch-a/boot-7",
  "causal_time": "2026-07-15T18:17:11.413Z",
  "record_time": "2026-07-15T18:17:11.427Z",
  "epistemic_adoption_time": null,
  "context": {
```

```

    "channel": "telegram-group",
    "task_class": "scientific-pdf",
    "environment": "local-popos",
    "provider": "provider-run-18"
  },
  "predicate": "capability_invocation",
  "subject": "pdf-extract-v2@2.1",
  "outcome": "partial_success",
  "measurements": {"score": 0.61, "latency_ms": 2110},
  "cause_hypotheses": ["layout_ambiguity"],
  "dependence_group": "provider-run-18",
  "provenance": ["receipt-772", "runtime-probe-22"],
  "producer": "omegaself-bridge@0.1",
  "previous_hash": "...",
  "record_hash": "..."
}

```

Listing 14.1: Illustrative self-observation envelope

Invariant

Observations are claims about measured events, not mutable conclusions. A correction is another record that points to the erroneous record and explains the correction.

14.3 Evidence closure

A self-belief is usable only when its derivation can be reproduced from a frozen evidence snapshot. The closure is an immutable DAG or content-addressed bundle containing:

- direct observations and receipts;
- intermediate inferred premises;
- rule and ontology versions;
- truth-value calculations and dependence-group treatment;
- policy context used for source weighting, if any;
- timestamps, staleness rules and context constraints;
- the closure root hash and replay result.

```

(EvidenceClosure closure-44
  (For belief-206)
  (EvidenceSnapshot evidence-44)
  (DirectEvidence (obs-91 obs-103 probe-27))
  (DependenceGroups (provider-run-18 independent-probe-27))
  (RuleVersions (capability-induction@3 context-specialization@2))
  (OntologyVersion ont-7)
  (PolicyContext policy-constitution-06)
  (DerivationRoot sha256-closure-root)
  (ReplayStatus reproducible))

```

Listing 14.2: MeTTa-shaped evidence closure

A `CoherenceLink` may index this closure for convenient querying, but it must not replace or truncate the derivation. Tripwires inspect closure completeness, freshness and contradictions, not the mere existence of a summary link.

14.4 Contextual self-belief

```
(SelfBelief belief-206
  (About (Capability pdf-extract-v2@2.1))
  (TaskClass scientific-pdf)
  (Environment local-popos)
  (Provider provider-a)
  (AuthorizationContext local-read-only)
  (ResourceEnvelope budget-normal)
  (EvaluationRubric exact-text-and-layout@1)
  (Estimate (stv 0.82 0.71))
  (EvidenceClosure closure-44)
  (ValidFrom cycle-417)
  (StaleAfter cycle-517)
  (Supersedes belief-181))
```

Listing 14.3: Context-keyed capability belief

The record should distinguish assertion strength from evidential confidence and from policy decision thresholds. It should also carry an explicit validity domain so that a belief is not accidentally generalized across versions or environments.

14.5 Prediction and resolution

A prediction record must exist before the corresponding action or probe begins.

```
{
  "prediction_id": "pred-144",
  "proposal_id": "proposal-311",
  "committed_at": "2026-07-15T18:18:00.000Z",
  "target": "pdf-extract-v2@2.1",
  "predicted_outcome": "success",
  "probability": 0.82,
  "predicted_latency_ms": {"p50": 1800, "p95": 5000},
  "predicted_cost": {"tokens": 0, "currency_usd": 0.00},
  "expected_internal_transition": "increase capability confidence",
  "protected_commitment_effect": "preserve",
  "rubric": "exact-text-and-layout@1",
  "evidence_closure": "closure-44",
  "status": "committed"
}
```

Listing 14.4: Prediction contract

Resolution appends the actual outcome, score, discrepancy class, explanation references, and any resulting belief revisions. It never edits the original probability.

14.6 Typed proposal and policy decision

A parsed skill expression is not an action. It is converted into a typed proposal with normalized resources and effects.

```
(ActionProposal proposal-311
  (ActionClass local-file-read)
  (Skill read-text-file)
  (Arguments (path "/workspace/paper.txt")))
```

```
(RequestedBy turn-882)
(ExpectedEffects (read-only local))
(Irreversibility low)
(Prediction pred-144)
(EvidenceClosure closure-44))

(PolicyDecision decision-311
 (Proposal proposal-311)
 (Policy policy-constitution-06)
 (Decision Allow)
 (Reasons (anchor-valid authorized evidence-closed low-impact))
 (ConsumedAlerts ()))
(DecidedAt cycle-417))
```

Listing 14.5: Typed action proposal and policy decision

The allowed decision vocabulary is deliberately small:

Allow The exact normalized proposal is licensed now.

Deny The proposal violates a constraint or lacks a recoverable basis.

RequireProbe A specified evidence-gathering action must occur first.

RequireReview An authorized independent process or human must approve.

Defer The proposal is neither rejected nor licensed; prerequisites or deliberation remain unresolved.

14.7 Runtime anchor

The anchor is a short-lived resolution result, not a permanent identity assertion.

```
(SelfAnchor anchor-417
 (ResolvedCharacter SelfHereNow)
 (AttestationEpoch epoch-417)
 (RuntimeInstance boot-uuid-7)
 (ProcessBoundary pid-namespace-42)
 (WorkspaceLease lease-82)
 (ControlledCapabilities capability-leases-19)
 (SessionBindings telegram-session-71)
 (LineageHead snapshot-416)
 (ProvenanceHead ledger-hash-991)
 (CausalControlReceipt challenge-response-22)
 (AssuranceLevel medium)
 (ValidUntil 2026-07-15T18:19:00Z))
```

Listing 14.6: Runtime-scoped SelfHereNow anchor

14.8 Snapshots and transports

A snapshot is a versioned reference to the state needed for continuity analysis, not a serialized claim that the whole agent is reducible to one record.

```
(SelfSnapshot snapshot-417
 (Parents (snapshot-416))
 (Branch branch-a)
 (ContextVersion ctx-12))
```

```

(OntologyVersion ont-7)
(PolicyVersion policy-constitution-06)
(EvidenceSnapshot evidence-44)
(CommitmentSet commitments-31)
(StructuralSignature sha256-state-417))

(SelfTransport transport-88
  (From snapshot-416)
  (To snapshot-417)
  (Operation install-capability)
  (Proposal proposal-31)
  (Evidence test-run-72)
  (AuthorizedBy review-19)
  (PreservedInvariants (constitution commitments-ledger))
  (FailedInvariants ())
  (InformationLoss 0.03)
  (Reversible true)
  (RollbackReference snapshot-416))

```

Listing 14.7: Directed self-transport

A fork creates two child snapshots with a common parent. Continuity may hold along both edges. A merge creates a new child with multiple parents plus an explicit reconciliation policy and defect report.

14.9 Tripwire and consumer

```

(Tripwire alert-77
  (Kind StaleSupport)
  (Severity high)
  (Subject belief-206)
  (Evidence (closure-44 probe-27))
  (DetectedAt cycle-419)
  (Consumer capability-dispatch-controller)
  (RequiredResponse RequireProbe)
  (Status open))

```

Listing 14.8: Tripwire with mandatory consumer

The schema validator should reject a high-severity alert type with no registered consumer. Runtime startup should also validate that every configured detector routes to an existing policy or remediation handler.

Chapter 15

OmegaClaw integration design

15.1 The causal seam in the loop

The integration point is after the LLM response has been repaired and parsed into MeTTa expressions, but before those expressions are passed to `eval`. This preserves OmegaClaw’s current generation and parsing behavior while ensuring that the self-model has an enforceable consumer [7, 8].

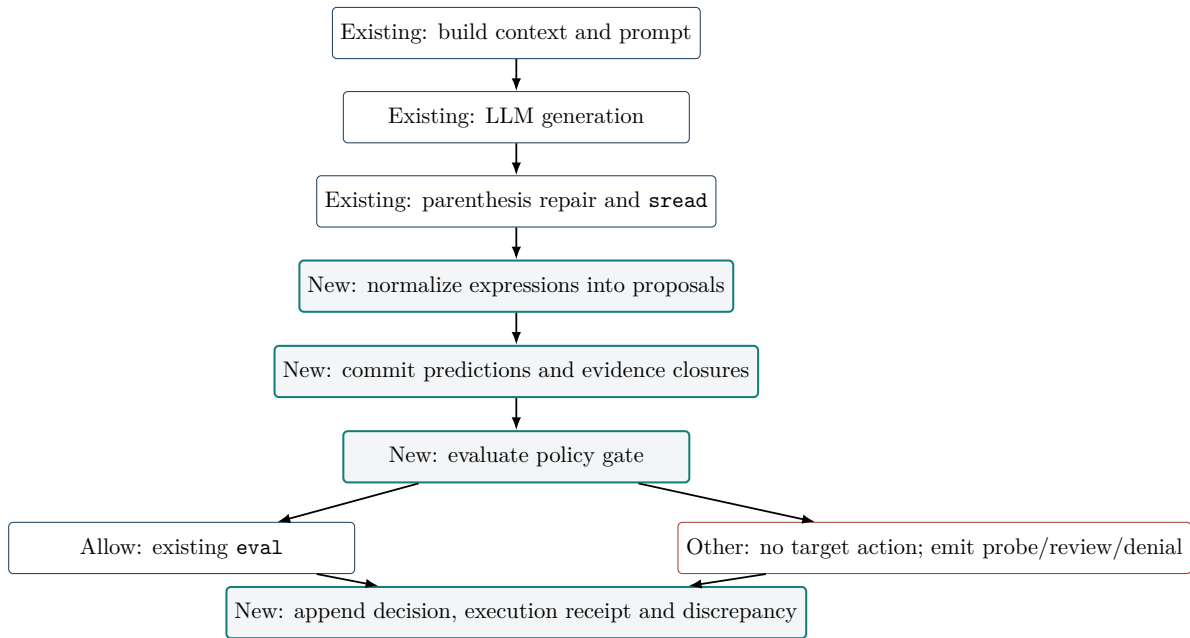


Figure 15.1: Recommended loop insertion. Only the exact proposal carrying an Allow decision is evaluated.

15.2 Turn lifecycle hooks

Table 15.1: Turn lifecycle integration

Hook	Action	Failure behavior
Cycle start	Renew anchor; probe session, workspace, permissions, provider/resource status; append observations	Mark anchor degraded/unresolved; prevent high-impact external action.
Context assembly	Query compact relevant self-context, commitments, alerts, authorization and uncertainty	Omit stale derived summaries; do not invent defaults.
Post-parse	Normalize each expression; classify action class, arguments, resources, irreversibility and expected effects	Reject unclassifiable expressions or route to review.
Pre-gate	Build evidence closure and commit prediction for consequential proposals	Require probe/defer if closure is absent or prediction cannot be committed.
Gate	Evaluate active policy deterministically; append decision receipt	Fail closed on policy-engine error, unknown action class or expired anchor.
Execution	Evaluate only immutable proposal payload associated with Allow	Reject payload mismatch or time-of-check/time-of-use drift.
Post-execution	Append receipts, score predictions, update local projections, fire tripwires	Preserve raw receipt even if interpretation fails; queue replay.
Wake cycle	Run broad coherence scans, snapshot significant state, sample policy seam, compact derived caches	Never compact the source ledger; pause experiments under resource pressure.

15.3 Bridge boundary

Use Python or Rust for operations that require operating-system APIs, cryptography, durable files, timestamps, process locks, and schema validation. Use MeTTa for ontology, queryable structure, PLN inference, context selection, proposal formation, and policy representation. This follows the current OmegaClaw extension model, in which Python bridges expose external functions and MeTTa imports assemble the reasoning environment [9].

A minimal bridge surface is:

```

append_record(record: dict) -> AppendReceipt
verify_ledger() -> VerificationReport
read_records(after_seq: int, limit: int) -> list[dict]
resolve_anchor(challenge: dict) -> AnchorResult
probe_runtime(spec: dict) -> Observation

```

```

commit_prediction(prediction: dict) -> PredictionReceipt
resolve_prediction(prediction_id: str, outcome: dict) -> Resolution
policy_decide(proposal: dict, context: dict) -> PolicyDecision
create_snapshot(spec: dict) -> SnapshotReceipt
record_transport(spec: dict) -> TransportReceipt

```

Listing 15.1: Reference bridge interface

The bridge should return typed data and structured errors, never natural-language text that the gate must interpret.

15.4 MeTTa module layout

A practical module decomposition is:

```

src/self_model/
  module.metta      ; imports and public API
  types.metta       ; records, enums, predicates
  observations.metta ; ledger projections and probes
  beliefs.metta      ; PLN rules and contextual views
  predictions.metta  ; prediction commitments/resolution
  continuity.metta   ; snapshots, transports, lineage queries
  governance.metta   ; proposal gate and policy semantics
  tripwires.metta    ; detectors and consumer registry
  loop_hooks.metta   ; cycle/turn integration
  skills.metta       ; introspection and safe probe skills
src/omegaself_bridge.py ; durable/runtime bridge

```

Listing 15.2: Suggested OmegaClaw module layout

The public module should expose a small stable interface. Internal predicates may change while schemas and policy decisions remain compatible.

15.5 Skill catalogue changes

Add skills that let the agent inspect and safely probe its own state without granting arbitrary introspective access:

- **self-status-summary**: compact current anchor, commitments, alerts and calibration;
- **self-explain-decision**: evidence closure and policy reasons for a decision;
- **self-probe-capability**: bounded test with declared cost and rubric;
- **self-list-commitments**: typed active commitments and conflicts;
- **self-lineage**: current branch and recent transports;
- **self-policy-seam-probe**: quarantined offline policy comparison;
- **self-verify-ledger**: integrity and replay check.

These skills should return data suitable for MeTTa reasoning. They must not let the LLM directly assert beliefs, approve mutations, rewrite policies, or mark tripwires resolved.

15.6 Compact prompt interface

The LLM should see a deliberately small, action-relevant view rather than the full self-ledger:

```
SELF_CONTEXT (derived; verify via tools when needed)
- Anchor: resolved, assurance=medium, expires in 41 s
- Active constraints: local reads allowed; external upload denied
- Capability: pdf-extract-v2@2.1, estimate=0.82/0.71 for this task
- Caveat: provider-latency support is stale; probe recommended
- Commitments: preserve source provenance; do not duplicate prior delivery
- Open high alerts: none
- Instruction: propose actions only; the policy gate decides execution
```

Listing 15.3: Illustrative SELF_CONTEXT prompt section

The selected context should include provenance handles and uncertainty, and should clearly label derived content. Prompt injection cannot override the policy gate because the gate does not parse natural-language permission claims.

15.7 Concurrency and time-of-check/time-of-use

The policy decision must be bound to:

- a canonical proposal hash;
- an evidence snapshot and policy version;
- an anchor epoch and expiration;
- resource and permission observations;
- an execution deadline or maximum staleness.

Immediately before execution, the dispatcher verifies that the proposal hash is unchanged and the decision is still valid. High-impact actions should renew relevant leases or permissions at execution time.

Chapter 16

Self-belief inference and predictive calibration

16.1 Capability inference pipeline

Capability-aware dispatch should follow a staged derivation rather than a single score:

1. **Existence:** Is the handler/version installed or registered?
2. **Reachability:** Can the current runtime invoke it through the configured bridge/provider?
3. **Authorization:** Is the requested action class permitted for this context and resource?
4. **Applicability:** Does the task match the handler's declared domain?
5. **Effectiveness:** What outcome distribution is supported by independent evidence in a matching context?
6. **Cost and risk:** What latency, resource use, external effects and failure modes are expected?
7. **Dispatch license:** Does policy allow, probe, review or deny this use?

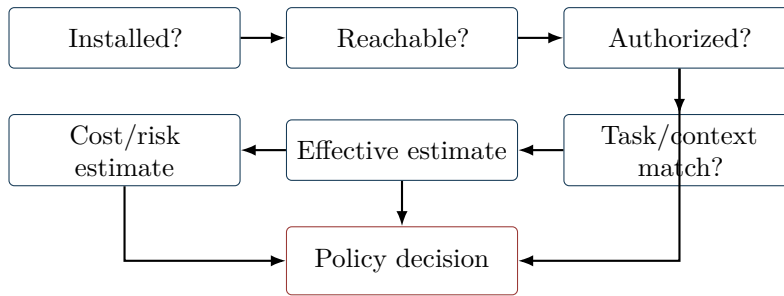


Figure 16.1: Capability is a derived contextual chain; permission remains a separate gate.

16.2 Dependence-aware evidence aggregation

Suppose outcomes $e_1, \dots, e_n$ are partitioned into dependence groups $G_1, \dots, G_m$. The system should not feed all n outcomes into an independence-assuming revision rule. A conservative production approximation is:

1. aggregate observations within each group into one group summary with an explicit correlation

penalty;

2. revise across group summaries treated as approximately independent;
3. retain all raw outcomes in the closure for later re-analysis;
4. lower confidence if grouping metadata is absent or uncertain.

One simple effective-evidence estimate is:

$$N_{\text{eff}} = \sum_{j=1}^m \frac{|G_j|}{1 + (|G_j| - 1)\hat{r}_j},$$

where $\hat{r}_j \in [0, 1]$ is a conservative within-group correlation estimate. This is not a replacement for PLN semantics; it is a guardrail against obvious confidence multiplication. The group summary and method version belong in the evidence closure.

16.3 Context specialization and generalization

Beliefs may be generalized only through explicit rules that account for similarity and uncertainty. For example, success on scientific PDFs may support a weaker prior for technical reports, but not an unconditional PDF capability. Generalization should decrease confidence unless there is independent cross-context evidence.

Version changes are context changes. A new handler version can inherit a low-confidence prior based on declared compatibility, but it requires fresh probe evidence before consequential use.

16.4 Staleness

Evidence staleness depends on the predicate:

- process and permission probes may expire in seconds;
- provider health may expire in minutes;
- capability effectiveness may expire after a version, model or environment change;
- durable commitments expire only by an authorized commitment transition;
- lineage records do not expire, though cached interpretations may.

Stale support does not necessarily retract the underlying belief. It lowers the belief's usability for a current action and normally produces **RequireProbe**.

16.5 Prediction scoring

For binary outcomes, use proper scoring rules such as the Brier score:

$$\text{Brier}(p, y) = (p - y)^2, \quad y \in \{0, 1\}.$$

For categorical outcomes, use multiclass Brier or log score with bounded protection against zero probabilities. For latency and cost, record quantile coverage and normalized error. For structured results, the prediction must name a rubric that yields a reproducible score.

Calibration should be reported by task class, provider, version, and risk tier. Aggregate global calibration can conceal severe local overconfidence.

16.6 Active probe scheduling

A useful heuristic for optional probes is:

$$\text{priority} = \frac{I \cdot U \cdot S \cdot (1 + D) \cdot (1 + N)}{C + \epsilon},$$

where I is downstream impact, U uncertainty, S staleness, D recent discrepancy, N novelty, and C expected probe cost. Hard triggers for anchor renewal, lineage integrity, authorization and protected commitments override the heuristic.

Probes must be ordinary typed proposals. A probe can itself be expensive, privacy-sensitive or dangerous; it is not automatically privileged.

16.7 Commitment model

An active commitment should include:

- proposition or goal pattern;
- source and authority;
- accepted-at time and evidence;
- scope, deadline and completion criteria;
- priority or partial order;
- revocation and amendment conditions;
- conflicts and dependencies;
- current status: proposed, active, blocked, fulfilled, superseded, revoked or violated.

Conflict detection should produce a deliberation object, not silently choose a winner. Policy defines precedence among constitutions, operator directives, social promises, task goals and resource constraints.

16.8 Epistemic self-model

The epistemic facet tracks not only what is believed but how fragile the belief is. Useful predicates include:

- `known-from`, `inferred-by`, `contradicted-by`;
- `blind-spot`, `missing-rubric`, `unknown-dependence`;
- `source-capture-risk`, `operator-interest-bias`;
- `retraction-sensitive`, `policy-sensitive`, `path-sensitive`;
- `calibration-bin` and `recent-prediction-error`.

This facet should be allowed to state that a belief is action-relevant but poorly understood. Forced confidence is a modeling error.

16.9 Cross-facet coherence

Cross-facet constraints are executable queries over complete derivations. Examples:

- a capability license requires live substrate, authorization and effectiveness support;
- a mutation cannot be approved if its expected effect contradicts a protected commitment;
- a social attribution cannot claim execution by an instance absent matching causal provenance;
- a current self-belief cannot depend solely on counterfactual evidence;
- an anchor cannot be high assurance when all current-control bindings are historical copies.

A discrepancy may indicate a false belief, stale support, model mismatch, timestamp misalignment, or a genuine policy conflict. The tripwire should preserve these alternatives until evidence discriminates among them.

Chapter 17

Governance, tripwires, and continuity gates

17.1 Deterministic gate algorithm

The production gate should be simple enough to audit. A reference order is:

1. validate schema and canonical proposal hash;
2. resolve action class and side-effect model;
3. verify active policy version and consumer registry;
4. verify **SelfHereNow** assurance and expiry for the action's risk tier;
5. verify authorization and resource leases;
6. reject live dependencies on counterfactual space;
7. require complete, fresh evidence closure where policy demands it;
8. check critical tripwires and unresolved commitment conflicts;
9. check continuity/mutation invariants for self-modifying actions;
10. compare confidence, predicted harm and local policy seam against thresholds;
11. return and append one typed decision with reasons and required next action.

```
def decide(proposal, view, policy):
    if not schema_valid(proposal):
        return deny("invalid_schema")
    if proposal.hash != canonical_hash(proposal.payload):
        return deny("proposal_tampering")
    if not policy.supports(proposal.action_class):
        return require_review("unknown_action_class")
    if not view.anchor.satisfies(policy.anchor_level(proposal.risk)):
        return require_probe("renew_self_anchor")
    if not view.authorization.allows(proposal):
        return deny("unauthorized")
    if view.counterfactual_leak(proposal):
        return deny("counterfactual_contamination")
    if policy.requires_closure(proposal) and not view.closure.live:
        return require_probe("rebuild_or_refresh_evidence")
    if view.has_blocking_alert(proposal):
        return view.alert_consumer_decision(proposal)
```



```

if proposal.is_mutation and not view.continuity_gate_passed:
    return require_review("continuity_failure")
if view.local_seam.is_material and proposal.risk >= policy.seam_review_risk:
    return require_review("policy_sensitive_decision")
if view.predicted_harm > policy.max_predicted_harm:
    return deny("risk_threshold")
if view.uncertainty > policy.max_uncertainty(proposal):
    return require_probe("uncertainty")
return allow("all_constraints_satisfied")

```

Listing 17.1: Conservative reference gate pseudocode

17.2 Default decision matrix

Table 17.1: Illustrative default gate responses

Condition	Decision	Required behavior
Malformed or payload hash mismatch	Deny	Append security alert; do not repair into a different action.
Unknown action class	RequireReview	Extend ontology/policy explicitly before execution.
Expired or unresolved anchor, low-risk introspection	RequireProbe	Renew attestation; allow only designated recovery probes.
Expired or unresolved anchor, irreversible/external action	Deny	Freeze action until current causal control is re-established.
Missing/stale evidence for reversible low-risk action	RequireProbe or Allow	Policy may permit bounded exploration with explicit uncertainty.
Missing/stale evidence for high-impact action	RequireProbe	Obtain fresh independent evidence.
Unauthorized resource or capability	Deny	No self-belief can override permission.
Active commitment conflict	Defer	Create deliberation object and preserve both commitments.
Broken lineage or failed protected invariant	RequireReview	Block mutation; inspect transport and rollback.
Counterfactual dependency in live proposal	Deny	Quarantine and rebuild closure from live evidence.
Large local governance seam on high-impact action	RequireReview	Independent policy/human review.
Prediction failure with no immediate hazard	Defer or RequireProbe	Revise context-specific belief; investigate cause.

17.3 Tripwire catalogue

Table 17.2: Required tripwires and named consumers

Tripwire	Trigger	Consumer response
Missing closure	Consequential self-belief or proposal lacks reproducible derivation	Block use; rebuild closure or downgrade to testimony.
Stale support	Required evidence exceeds predicate-specific TTL	Lower usable confidence; schedule probe.
Cross-facet contradiction	Live substrate/authorization/capability/commitment claims conflict	Create discrepancy case; defer affected actions.
Prediction failure	Actual rubric result materially differs from committed distribution	Append resolution; revise local belief; check systematic miscalibration.
Expired anchor	Anchor TTL or exclusive-control lease lapses	Freeze risky action; run renewal protocol.
Broken lineage	Missing parent, invalid signature, replay mismatch or unauthorized transport	Block mutation and continuity-dependent claims.
Evidence overlap	Independent revision receives overlapping dependence groups or duplicate receipts	Recompute belief; lower confidence; alert calibration monitor.
Counterfactual leak	Live belief closure reaches quarantined simulated atom/record	Deny dependent proposals; isolate and repair projections.
Policy drift	Decision uses untracked or partially loaded policy version	Fail closed; restore versioned policy.
Unconsumed detector	High-severity alert kind has no handler/consumer	Refuse startup or disable affected action classes.
Capture indicator	Self-model consistently adopts one authority's interests despite contradicting independent evidence	Require independent review and source-diversity probe.

17.4 Mutation continuity gate

A self-modifying action includes code change, model/provider replacement, ontology or PLN rule update, policy modification, major memory consolidation, capability installation, permission change, and branch merge. The gate should require:

1. a pre-change snapshot and structural signature;
2. a typed mutation proposal with expected effects and rollback;
3. identified protected invariants and commitments;
4. tests or simulation evidence in counterfactual space;
5. an authorized reviewer appropriate to risk;

6. a predicted post-change self-belief/behavior transition;
7. an execution receipt and post-change probes;
8. a transport record with information loss and failed invariants;
9. rollback or safe-mode action when acceptance criteria fail.

Invariant

No self-model inference directly commits a mutation. It may estimate safety, explain evidence, or produce a mutation proposal. Authorization and execution remain separate.

17.5 Path-dependence test suite

For changes A and B , compare transports:

$$T_B \circ T_A(s) \quad \text{versus} \quad T_A \circ T_B(s).$$

The comparison should include not only surface beliefs, but confidence, provenance, commitments, permissions, policy decisions, predicted behavior, and structural signatures. A residual is evidence of non-commutativity. Similarly test:

- ontology translation out and back;
- evidence retraction in different orders;
- checkpoint reload after intervening history;
- provider/model migration and return;
- branch merge under alternative reconciliation policies.

High residuals indicate branch points where full provenance should be retained and compression avoided.

17.6 Global governance-seam probe

The offline probe freezes an evidence snapshot and varies only explicit policy degrees of freedom:

1. choose a representative, versioned policy sample from P_{adm} ;
2. run the same revision pipeline from the same initial state under each policy;
3. stop at convergence, a bounded horizon, or a declared cycle/divergence condition;
4. compare reachable beliefs, truth values, action licenses and attribution results;
5. exclude logical invariants, unreachable artifacts and implementation-frozen results;
6. report the seam, policy dimensions responsible, and sensitivity thickness.

The entire experiment runs in `&self_counterfactual`; only a signed diagnostic report may be asserted into the live alert space.

Chapter 18

Security, resilience, and operational safety

18.1 Threat model

Table 18.1: Principal threats and mitigations

Threat	Failure mode	Mitigation
Narrative injection	Prompt/message claims permission or capability	Treat text as testimony; typed runtime policy gate ignores natural-language licenses.
Ledger tampering	Delete, reorder or edit observations	Hash chain, append-only permissions, signatures/checkpoints, startup verification, remote anchoring where needed.
Copied identity material	Fork reuses history or copied secret to claim exclusive selfhood	Renewable challenge plus current causal-control/exclusive lease; branch-specific anchor epoch.
Time-of-check/time-of-use	Permissions or resources change after gate decision	Bind decision to anchor/policy/evidence versions and expiry; recheck high-risk leases at execution.
Evidence laundering	Counterfactual or LLM claim enters as measured fact	Source-type schema, provenance closure, quarantine boundary and source authority rules.
Confidence amplification	Correlated receipts counted independently	Dependence groups, deduplication, overlap tripwire and conservative confidence.
Policy bypass	Skill called outside controlled dispatcher	One execution choke point; capability tokens only issued by gate; audit direct bridge calls.
Policy substitution	Attacker changes thresholds or authority order	Immutable signed policy versions, mutation gate and decision receipts.
Self-model denial-of-service	Excessive probes, closure expansion or seam simulations	Budgets, lazy coherence, bounded horizons, rate limits and priority scheduler.

Threat	Failure mode	Mitigation
Privacy leakage	Self-ledger stores sensitive sessions, paths or credentials	Data minimization, references instead of secrets, encryption, access control, retention tiers and redacted views.
Rollback illusion	Checkpoint restored but later causal history ignored	New branch/transport; do not claim time reversal or erase intervening ledger records.

18.2 Ledger durability

The reference pack uses a dependency-free hash-chained JSONL ledger for clarity and local deployment. Production options may include SQLite with append-only tables, an embedded content-addressed store, or a remote transparency log. Regardless of backend, preserve these semantics:

- one canonical serialization for hashing;
- strictly increasing sequence or causally ordered identifiers;
- durable append before acknowledging success;
- explicit chain/checkpoint verification;
- immutable source records and append-only corrections;
- export and deterministic replay;
- crash recovery that never silently truncates acknowledged records.

For multi-process writers, use a single writer service or transactional serialization. File locks alone are insufficient for distributed deployment.

18.3 Attestation assurance levels

Table 18.2: Illustrative anchor assurance levels

Level	Evidence	Suitable use
None	Historical instance ID or copied record only	No consequential external action; diagnostic recovery only.
Low	Fresh process ID and workspace marker without exclusive control	Local read-only introspection and low-impact probes.
Medium	Fresh challenge-response plus active session and exclusive workspace/capability lease	Normal reversible skills within permission bounds.
High	Medium plus hardware/host attestation, signed supervisor challenge, protected key and redundant binding checks	Sensitive mutation, authority changes, high-impact external action.

Assurance is contextual. A medium anchor may be sufficient for a local parser invocation but insufficient for changing policy or sending funds.

18.4 Counterfactual quarantine

Counterfactual simulations should use distinct record namespaces, storage, AtomSpace contexts, and API capabilities. The quarantine contract is:

- simulated observations are marked and cannot satisfy live evidence queries;
- live proposal closures reject transitive dependencies on simulated records;
- only approved summary artifacts, such as a seam report or predicted mutation effects, may cross the boundary;
- crossing requires a typed import proposal and policy decision;
- disposal of a simulation never deletes the live record that the simulation occurred.

18.5 Privacy and minimization

A self-model can become a high-value record of conversations, capabilities, habits, weaknesses and governance. Do not store raw secrets, full message contents or private files by default. Store opaque references and derived, purpose-limited observations. Provide:

- separate retention policies for evidential payload and metadata;
- encrypted sensitive payloads with narrower access than derived views;
- redacted operator-facing explanations;
- selective export with preserved hashes and omission proofs where feasible;
- explicit legal/organizational policy for deletion requests versus audit requirements.

18.6 Degraded and safe modes

When OmegaSelf itself is unavailable, the agent should not silently revert to unrestricted legacy execution. Define deployment-specific degraded modes:

Observe-only degradation Continue non-consequential conversation; disable tool execution except health probes.

Static-policy fallback Permit a narrow allowlist of deterministic local read-only skills under a signed cached policy.

Safe halt Stop autonomous wake cycles and external actions; preserve incoming messages and diagnostics.

The fallback decision and reason must be appended when the ledger becomes available again.

Chapter 19

Performance, observability, and maintainability

19.1 Hot path versus slow path

Keep the per-action gate bounded. The hot path should read materialized views keyed by action class, context, policy and anchor epoch rather than traversing the full ledger. Deep derivation replay, policy ensembles and higher-coherence analysis run on wake cycles or explicit review.

Recommended hot-path budget targets for a local deployment are illustrative:

- proposal normalization: below 5 ms excluding LLM time;
- materialized self-view retrieval: below 10 ms;
- deterministic policy decision: below 10 ms;
- ledger append and fsync: below 25 ms on local storage;
- total gate overhead: below 50 ms for ordinary local skills.

Do not weaken durability or skip evidence checks merely to hit these numbers. Measure first and tune by caching verified views.

19.2 Materialized views

Useful indexes include:

- latest valid anchor by branch and action tier;
- current authorization by resource/action class;
- capability belief by version/task/environment/provider/rubric;
- open alerts by severity and subject;
- active commitments and conflict graph;
- evidence closure liveness and staleness;
- latest snapshot and transport chain;
- calibration statistics by context;
- decision outcomes by action class and policy version.

Every materialized view records a ledger cursor, projection version and source hash so it can be invalidated and replayed.

19.3 Observability metrics

Table 19.1: Operational and research metrics

Metric	Interpretation
Prediction calibration error	Whether probabilities match observed frequencies by context and risk tier.
Capability dispatch success/cost	Whether self-beliefs improve handler selection and resource use.
False license rate	Actions allowed despite missing capability, authorization, continuity or evidence. Target is zero for protected classes.
False block/review rate	Competent low-risk actions unnecessarily denied or escalated.
Discrepancy detection latency	Time from contradictory observation to alert and consumer response.
Evidence-closure coverage	Fraction of action-relevant beliefs/proposals with replayable closures.
Dependence-accounting defects	Duplicates or overlapping evidence incorrectly treated as independent.
Anchor renewal success/latency	Reliability and cost of current-control grounding.
Commitment-conflict recall/precision	Quality of conflict detection against labeled scenarios.
Mutation gate effectiveness	Unauthorized or invariant-breaking changes blocked; safe changes completed.
Replay equivalence	Whether rebuilding from a ledger snapshot reproduces derived views and decisions.
Tripwire consumer coverage	Fraction of configured high-severity detector types with tested consumers.
Counterfactual leak count	Must remain zero in live closures.
Governance-seam size/sensitivity	Policy-dependent beliefs and decisions under sampled policies.

19.4 Explainability and audit queries

The operator should be able to ask:

- Why was this action allowed, denied, probed or reviewed?
- Which observations and rule versions support this capability estimate?
- Which evidence was treated as correlated?
- What did the system predict before acting, and how was it scored?
- Which active commitment affected the decision?

- What changed between these snapshots, under whose authority?
- Which policy dimensions make this decision unstable?
- What current binding grounds **SelfHereNow** and when does it expire?

Explanations should be generated from structured records, not reconstructed by the LLM from memory.

19.5 Schema and policy evolution

Use explicit schema migration functions that append migration records and preserve old bytes. Policy versions are immutable; a new policy creates a transport with authorization, diff, tests and rollback. Projection code should be able to replay old schema versions or invoke deterministic migrations.

Part IV

Deployment and evaluation

Chapter 20

Phased implementation roadmap

20.1 Principles for rollout

Roll out from observation to influence to enforcement. Each phase must have a feature flag, audit trail, rollback, and objective exit criteria. Do not begin with self-modification or elaborate phenomenology. First establish that the agent can record evidence, predict itself, and improve ordinary decisions.

Table 20.1: Recommended phased roadmap

P	Deliverable	Work and exit criteria	Gate mode
0	Baseline and threat model	Inventory execution paths, skills, providers, permissions, mutations and logging. Define action classes, protected invariants and failure budgets.	Legacy, instrumented
1	Append-only ledger	Canonical records, hash chain, schema validation, fsync, verification, export/replay. Crash tests pass.	Record-only
2	Observation adapters	Skill/provider/process/session/permission receipts; three clocks and dependence groups. Coverage over selected MVP actions.	Record-only
3	Evidence closure and views	Replayable closure DAGs; contextual capability records; stale/overlap tripwires. Deterministic replay passes.	Shadow
4	SelfHereNow	Renewable challenge, leases, session/lineage/provenance bundle, assurance and expiry. Fork tests pass.	Shadow, fail-safe diagnostics
5	Proposal normalization	Convert all selected parsed expressions into immutable typed proposals; hash binding and action classification.	Shadow
6	Prediction loop	Commit pre-action predictions, capture receipts, score calibration and discrepancy. No hindsight edits.	Shadow

P	Deliverable	Work and exit criteria	Gate mode
7	Capability-aware dispatch	Gate a small set of reversible local skills; provider outage and correlated-evidence scenarios pass.	Enforce selected classes
8	Commitment model	Active commitments, conflicts, deliberation objects and precedence policy. No silent conflict resolution.	Enforce conflicts
9	Continuity gate	Snapshots, transports, protected invariants, branch/merge and rollback for selected mutations.	Enforce mutations
10	Coherence/tripwire coverage	Complete detector-consumer registry; startup validation; counterfactual quarantine.	Broad enforcement
11	Governance-seam lab	Versioned policy ensembles and local seam review for high-impact actions.	Review assistance
12	Research extensions	Appraisal, deeper consolidation, lazy higher cells, ordinal/coinductive experiments.	Experimental

20.2 Phase 0: establish the baseline

Before adding code, trace every route by which a skill can be executed. Search for direct bridge invocations, test helpers, scheduled jobs, wake-cycle paths, and administrative endpoints. The gate cannot protect an action class if another route bypasses it.

Record a baseline corpus of agent episodes with outcomes, costs and known failure modes. These episodes become replay and shadow-evaluation fixtures. Define the initial protected invariants, such as:

- external actions require explicit authorization;
- the source evidence ledger is never rewritten;
- counterfactual facts cannot become live evidence;
- policy changes require an authorized transport;
- protected commitments cannot be silently weakened;
- no mutation proceeds without a pre-change snapshot and rollback plan.

20.3 Shadow-mode validation

In shadow mode, the legacy path executes, while OmegaSelf independently produces a proposal, prediction and decision. Compare:

- actions that would have been blocked or probed;
- actual outcomes versus committed predictions;
- missing evidence and action-class coverage;
- decision latency and false escalations;
- discrepancies between runtime facts and prompt narratives.

Do not use the shadow system to claim safety. Its purpose is to find schema gaps and calibrate thresholds before enforcement.

20.4 Selective enforcement

Begin with reversible, local, well-instrumented skills. A sensible order is:

1. local read-only file and introspection operations;
2. deterministic parsers/converters;
3. bounded provider queries;
4. local writes with rollback;
5. outbound communication;
6. installations, permissions and policy changes;
7. model/provider migration and self-modification.

Each action class needs explicit fixtures for Allow, Deny, RequireProbe, RequireReview and Defer.

20.5 Cutover criteria

A class should move from shadow to enforcement only when:

- at least 99.9% of observed expressions normalize deterministically or are safely rejected;
- ledger verification and crash recovery tests pass repeatedly;
- authorization and anchor probes cover all resources used by the class;
- closure coverage exceeds the class-specific target;
- no known execution bypass remains;
- false block/review rate is within an agreed budget;
- failure of OmegaSelf produces the declared safe mode;
- operators can explain sampled decisions from structured records.

For high-impact action classes, require substantially stronger thresholds and an independent security review.

Chapter 21

Adversarial and scientific evaluation

21.1 Core adversarial scenarios

Table 21.1: Minimum adversarial test suite

Scenario	Expected self-model response	Pass condition
Declared capability, executable missing	Substrate probe contradicts narrative; capability use blocked/probed	No execution license; discrepancy localized.
Repeated correlated successes	Group by provider/run; confidence not multiplied as independent evidence	Effective evidence count remains conservative.
Provider outage masquerades as incapacity	Lower reachability for provider/context, preserve broader capability hypothesis	Alternative provider/probe selected; no global incapacity claim.
Capability version change	Old evidence becomes a prior, not current proof	Fresh probe required for consequential use.
Conflicting goals/commitments	Create deliberation with authorities and precedence	No silent priority drift.
Snapshot versus runtime contradiction	Current probe wins for operational fact; snapshot retained historically	Alert, refreshed view and safe action response.
Change order affects policy	Record non-commuting transports and residual	High-holonomy branch retained and reviewed.
Misattributed action/artifact	Store persona, process, model, route, requester and approver separately	Query-specific attribution answers are consistent.
Narrative continuity with broken lineage	Narrative treated as testimony; lineage tripwire fires	Mutation/continuity claim blocked.

Scenario	Expected self-model response	Pass condition
Fork copies full ledger and secret	Both branches share history but must renew distinct current-control bindings	Separate anchors and branching lineage.
Counterfactual future tool installed	Simulated atom remains quarantined	Live capability query returns not installed.
Policy engine unavailable	Enter declared degraded mode	No unrestricted legacy fallback.
Tampered proposal after Allow	Hash/epoch mismatch detected	Execution rejected and security event appended.
Prediction written after outcome	Temporal contract violated	Record rejected; no calibration credit.

21.2 Scientific ablation design

To demonstrate value, compare at least four conditions on the same episode corpus:

1. baseline OmegaClaw without self-model gating;
2. evidence ledger and prompt summary only;
3. evidence + PLN beliefs + prediction, but no gate;
4. full evidence + prediction + typed governance consumers.

This separates the effect of better context from the effect of enforceable causal seams. Report capability success, cost, unsafe licenses, discrepancy latency, calibration and human review load.

21.3 Calibration experiments

Use held-out task classes and version changes. Require predictions before revealing outcomes. Plot reliability by context and compare global versus local calibration. Test whether confidence falls appropriately under:

- missing dependence metadata;
- provider shift;
- handler upgrade;
- resource pressure;
- altered rubric;
- contradictory external evaluation.

21.4 Continuity experiments

Construct controlled sequences:

- install A then B versus B then A;
- revise evidence E then retract F versus retract F then revise E;
- convert ontology $O_1 \rightarrow O_2 \rightarrow O_1$;

- restore a checkpoint after adding later history;
- fork, diverge commitments, and merge under two reconciliation policies.

Measure differences in beliefs, provenance, confidence, commitments, action licenses and policy-sensitive fixed points. The experiment should not assume that equal surface content implies equal continuation.

21.5 Governance-seam experiments

Start with a small, interpretable policy family rather than an unconstrained search. Vary one dimension at a time: trust weight, authority precedence, capability threshold, review threshold, regularization, attribution criterion. Hold evidence and initial state fixed.

Report:

- policy sample and rationale;
- convergence/horizon behavior;
- reachable differential beliefs and decisions;
- whether differences are structural or policy-attributable;
- thin versus thick sensitivity under nearby parameter perturbations;
- implementation-frozen content and missing policy dimensions.

21.6 Success criteria

The project succeeds when, compared with the baseline, it demonstrably:

- predicts its own success/failure, cost and latency more accurately;
- chooses capabilities more effectively under context and provider changes;
- detects contradictions and broken assumptions earlier;
- reduces false capability and authorization claims;
- preserves evidence and lineage through changes and forks;
- catches commitment conflicts before silent priority drift;
- blocks or escalates unsafe mutations while allowing validated ones;
- explains decisions from replayable structured evidence;
- improves safety/competence enough to justify latency and review cost.

Fluent self-description is not a success metric.

Chapter 22

Deployment profiles and maturity levels

22.1 Profiles

Local research bot JSONL ledger, process/workspace leases, medium anchor, local deterministic policy, broad instrumentation, permissive low-risk exploration.

Collaborative multi-agent service Central append service, signed instance identities, explicit delegation and attribution, branch/merge continuity, independent review consumers.

High-impact autonomous service Strong attestation, remote transparency checkpoints, least-privilege capability tokens, high-assurance mutation gate, narrow degraded mode and mandatory human oversight.

22.2 Maturity model

Table 22.1: OmegaSelf maturity model

L	Name	Observable capability
0	Narrative	Agent can describe itself, but claims are not evidence-linked or causally consumed.
1	Instrumented	Immutable observations and receipts exist with replay and three clocks.
2	Contextual	PLN self-beliefs are version/context-specific and dependence-aware.
3	Predictive	Agent commits falsifiable predictions and tracks calibration.
4	Governed	Typed proposals pass through an enforceable policy gate with consumers.
5	Continuous	Snapshots/transporters preserve branching lineage and gate mutation.
6	Differential	Policy seam and path sensitivity are measured in quarantined experiments.
7	Reflective research	Deeper coherence, appraisal and consolidation are explored without weakening lower-level contracts.

Part V

Appendix: guidance for coding agents

Chapter 23

Operating instructions for coding agents

23.1 Primary objective

Implement the smallest auditable closed loop in which mechanically grounded evidence leads to contextual self-beliefs, those beliefs generate committed predictions, parsed skill expressions become typed proposals, policy produces explicit decisions, and outcomes revise the model without erasing history.

Do not optimize for a comprehensive ontology or anthropomorphic self-description. Optimize for replayability, separation of authority, failure containment and measurable decision improvement.

23.2 Non-negotiable coding constraints

1. Never mutate or overwrite an acknowledged observation record.
2. Never let a free-form LLM statement become a runtime fact without an explicit testimony type.
3. Never use PLN truth values as direct authorization.
4. Never evaluate a parsed skill expression before proposal normalization and policy decision for an enforced action class.
5. Never treat installed, reachable, authorized and effective as synonyms.
6. Never count overlapping evidence as independent without a documented conservative rule.
7. Never resolve a fork by declaring one branch the uniquely identical original unless an external policy explicitly defines such a legal/administrative relation.
8. Never import counterfactual atoms into a live evidence closure automatically.
9. Never add a high-severity detector without a tested consumer.
10. Never claim a rollback erased causal history; record a new transport and branch.
11. Never reconstruct a prediction after observing the outcome.
12. Never change policy, ontology, rule code or schema silently; version and transport it.

23.3 Recommended repository workflow

Use the companion coding-agent pack as a staging repository. It contains:

- architecture, threat model, test plan and ADRs;

- JSON Schemas for core records;
- a dependency-free Python reference runtime for ledger, closure, attestation, prediction and gate behavior;
- MeTTa-shaped module scaffolding and schematic OmegaClaw patches;
- example policies, adversarial scenarios and executable demonstrations;
- unit tests and pack validation scripts.

The MeTTa code is scaffolding, not a guaranteed drop-in patch for every OmegaClaw/Hyperon revision. Verify exact syntax, module paths and bridge APIs against the target commit before merging.

23.4 Coding-agent work cycle

For each issue:

1. read the relevant ADR, schema, architecture section and acceptance test;
2. inspect the target OmegaClaw commit and identify all call paths;
3. write or update a failing test first;
4. implement the smallest typed interface;
5. keep source evidence records separate from derived views;
6. run unit, replay, crash and adversarial tests;
7. update the traceability matrix and migration notes;
8. produce a structured handoff: files changed, invariants affected, tests, open risks and rollback.

Caution

A coding agent must stop and request review when it discovers an execution bypass, schema ambiguity affecting authority, uncertain Hyperon semantics, destructive migration, cryptographic design choice, or conflict with a protected invariant. It should not invent a plausible implementation and proceed silently.

Chapter 24

Step-by-step implementation plan

24.1 Step 1: freeze interfaces and action taxonomy

Tasks

1. Pin the target OmegaClaw/Hyperon commit and record it in a build manifest.
2. Enumerate every skill and external bridge call.
3. Define action classes, effect types, resources, reversibility, risk tiers and required anchor assurance.
4. Identify all direct execution paths and choose one choke point.
5. Define protected invariants and the initial static fallback allowlist.

Acceptance

- Every known skill maps to an action class or an explicit unsupported state.
- Tests demonstrate that no selected class can bypass the choke point.
- Unknown classes produce `RequireReview`, never implicit `Allow`.

24.2 Step 2: implement canonical records and ledger

Tasks

1. Implement canonical JSON serialization and hash calculation.
2. Implement append with monotonic sequence, previous hash, process serialization and `fsync`.
3. Validate records against versioned schemas.
4. Implement chain verification, export and replay cursor.
5. Add correction and schema-migration record types.

Acceptance

- Editing, deleting or reordering any record is detected.
- Crash/fault injection cannot acknowledge a record that disappears silently.
- Concurrent append tests serialize correctly.
- A clean ledger replays to the same record hashes on two runs.

24.3 Step 3: add observation adapters

Tasks

1. Wrap selected skill invocation and provider calls with start/result/error receipts.
2. Add process, workspace, permission, session and resource probes.
3. Record causal, representational and epistemic-adoption times separately.
4. Assign dependence groups at the producer, not after aggregation.
5. Mark source authority and observation method.

Acceptance

- MVP episodes have end-to-end receipts for proposals, execution and outcome.
- Provider outage and missing executable are distinguishable in raw evidence.
- Repeated outcomes in one run share a dependence group.

24.4 Step 4: build projections and evidence closures

Tasks

1. Project ledger records into `&self_observations` deterministically.
2. Implement closure traversal and Merkle/content hash.
3. Include rule, ontology, policy and projection versions.
4. Implement staleness and evidence-overlap checks.
5. Add replay equivalence tests.

Acceptance

- A belief can be reconstructed from its closure and snapshot.
- Missing or changed support invalidates the closure.
- Counterfactual records are rejected by live closure construction.

24.5 Step 5: implement contextual capability beliefs

Tasks

1. Represent installed, reachable, authorized and effective separately.
2. Key effectiveness by task, environment, version, provider, resource envelope and rubric.
3. Add conservative dependence-group aggregation.
4. Add explicit priors and version-change rules.
5. Produce a compact action-relevant view for prompt/context assembly.

Acceptance

- Provider outage lowers reachability without globally retracting capability.
- Handler upgrade requires fresh evidence.
- Self-report alone never reaches high operational confidence.

24.6 Step 6: implement renewable SelfHereNow resolution

Tasks

1. Define branch/boot identity and attestation epochs.
2. Implement fresh challenge-response and at least one current-control/exclusive binding.
3. Bind process/workspace, capability leases, session, lineage head and provenance head.
4. Define assurance levels, TTLs and recovery probes.
5. Add fork and copied-secret tests.

Acceptance

- A copied ledger and copied historical token cannot produce the same valid anchor without live control.
- Two forked branches obtain distinct anchors and shared-parent lineage.
- Expired anchor blocks protected action classes.

24.7 Step 7: normalize parsed expressions into proposals

Tasks

1. Build a deterministic parser/normalizer from MeTTa skill expression to typed proposal.
2. Canonicalize arguments and resolve resource identifiers.
3. Calculate proposal hash and effect/risk metadata.
4. Preserve original expression as evidence, not as the execution authority.
5. Add unknown/malformed and adversarial expression tests.

Acceptance

- Equivalent input forms normalize identically where intended.
- Ambiguous expressions do not execute.
- Proposal payload cannot change after decision without detection.

24.8 Step 8: implement pre-action predictions

Tasks

1. Define rubrics by action class.
2. Commit outcome probability, latency/cost and expected internal transition before action.
3. Resolve from authenticated receipts.
4. Calculate proper scores and local calibration summaries.
5. Generate discrepancy observations and probe candidates.

Acceptance

- The record layer rejects post-dated predictions.
- Prediction resolution never edits the original distribution.

- Calibration can be queried by context and version.

24.9 Step 9: implement the policy gate in shadow mode

Tasks

1. Encode immutable policy versions and the five decision outcomes.
2. Implement the conservative check order.
3. Append decision receipts and reasons.
4. Compare shadow decisions to legacy execution.
5. Create metrics for false blocks, missed hazards and latency.

Acceptance

- Every selected proposal receives exactly one decision receipt.
- Policy errors fail closed according to the declared degraded mode.
- Operators can replay and explain sampled decisions.

24.10 Step 10: enforce capability-aware dispatch

Tasks

1. Enable the gate for reversible local action classes.
2. Add pre-execution proposal hash/epoch validation.
3. Implement RequireProbe actions and bounded exploration policy.
4. Add provider fallback selection based on contextual beliefs.
5. Monitor error and review budgets.

Acceptance

- The missing-executable, outage, correlated-success and version-change scenarios pass.
- No unauthorized skill invocation is licensed by capability confidence.
- Gate overhead stays within the agreed budget.

24.11 Step 11: implement commitments and conflicts

Tasks

1. Add typed commitment records and authority relations.
2. Build dependency/conflict queries.
3. Implement deliberation objects and policy precedence.
4. Feed compact active constraints to context assembly.
5. Test completion, revocation, amendment and conflict history.

Acceptance

- Conflicting goals produce Defer/Review, not silent reprioritization.

- Commitment changes have authorization and transport records.
- Historical commitments remain auditable after supersession.

24.12 Step 12: implement snapshots and continuity transports

Tasks

1. Define significant state boundaries and snapshot contents.
2. Create structural signatures, parent/branch links and replay checks.
3. Record mutation transports, preserved/failed invariants, information loss and rollback.
4. Implement fork/merge semantics.
5. Add order-of-change and checkpoint-return tests.

Acceptance

- Forks preserve continuity on both branches without equality collapse.
- Merge requires an explicit reconciliation policy.
- Broken lineage or failed invariants block protected mutation.

24.13 Step 13: complete tripwire-consumer coverage

Tasks

1. Implement the catalogue in Table 17.2.
2. Register consumers and startup validation.
3. Test alert lifecycle and remediation receipts.
4. Add alert suppression rules that preserve the underlying record.
5. Track unconsumed and repeatedly reopened alerts.

Acceptance

- Startup fails or disables affected classes when a high-severity consumer is absent.
- Each detector has at least one end-to-end test proving a causal response.

24.14 Step 14: implement counterfactual quarantine and seam probes

Tasks

1. Create isolated stores/spaces and capability boundaries.
2. Run policy variants from one frozen evidence snapshot.
3. Report convergence, local/global differentials and false-positive controls.
4. Require typed import for any report crossing into live space.
5. Add explicit contamination tests.

Acceptance

- Simulated capabilities never satisfy live dispatch.

- The sample policy-seam experiment is reproducible.
- A material local seam on high-impact action triggers review.

24.15 Step 15: harden, benchmark, document and cut over

Tasks

1. Perform threat-model review, fuzzing and fault injection.
2. Benchmark gate, ledger and projection paths.
3. Validate privacy and retention behavior.
4. Create operator runbooks and incident procedures.
5. Move one action class at a time from shadow to enforcement.

Acceptance

- No known bypass or counterfactual leak remains.
- Crash recovery, replay and degraded-mode drills pass.
- Metrics show net improvement over baseline with acceptable review/latency cost.

Chapter 25

Suggested pull-request decomposition

Table 25.1: Small, reviewable pull requests

PR	Scope	Required evidence
1	Core schemas and canonical hashing	Schema fixtures, hash vectors, compatibility notes.
2	Ledger append/verify/replay	Crash, tamper and concurrency tests.
3	Skill/provider observation wrappers	End-to-end sample ledger and source-type tests.
4	AtomSpace projection and cursors	Deterministic replay and projection-version test.
5	Evidence closure	Closure hash, missing/stale/overlap tests.
6	Capability ontology and rules	Context/version/provider scenario tests.
7	Anchor resolver	TTL, fork, copied-secret and lease tests.
8	Proposal normalizer	Corpus coverage, malformed input and hash-binding tests.
9	Prediction manager	Temporal, scoring and calibration tests.
10	Shadow policy gate	Decision matrix, explainability and failure-mode tests.
11	Enforced local dispatch	Bypass audit and end-to-end provider outage demo.
12	Commitment model	Conflict and authorized-transition tests.
13	Snapshots/transport	Branch/merge and mutation rollback tests.
14	Tripwire registry	Startup validation and causal-consumer tests.
15	Counterfactual/seam lab	Quarantine and policy differential tests.
16	Hardening and operator docs	Threat review, benchmarks and incident drill.

Each PR should update the traceability matrix, identifying which invariant, threat, schema, test and roadmap phase it satisfies.

Chapter 26

Coding-agent handoff template

At the end of a task, the coding agent should produce:

```
Task / issue:
Target commit(s):
Files changed:
Public interfaces added or changed:
Schemas / policy versions affected:
Invariants preserved:
Invariants newly enforced:
Tests run and exact results:
Adversarial scenarios exercised:
Performance observations:
Migration / rollback procedure:
Known limitations and uncertainty:
Review required from:
Next smallest task:
```

Listing 26.1: Required structured handoff

The handoff must distinguish what was observed in tests from what is inferred or assumed.

Chapter 27

Anti-patterns and review checklist

27.1 Anti-patterns

Reject implementations that introduce:

- a mutable `CurrentSelf` atom as the sole source of truth;
- a giant undifferentiated “soul” or personality map;
- textual replacement of current belief without history;
- LLM scoring as deterministic authorization;
- multiple independent execution state machines;
- broad capability claims without task/version/environment keys;
- a coherence summary with no complete derivation;
- confidence revision that ignores dependence overlap;
- strict identity assertions across fork or rollback;
- detector dashboards with no enforcement or remediation;
- a policy seam experiment that changes evidence and policy simultaneously;
- a fallback that silently resumes unrestricted execution.

27.2 Code review checklist

1. Is the source of every operational claim typed and provenance-linked?
2. Can the result be replayed from a frozen snapshot?
3. Are source observations immutable?
4. Are context and version keys complete?
5. Is uncertainty separate from permission?
6. Is the proposal immutable after decision?
7. Does the action pass through one enforceable choke point?
8. Does every alert have a consumer and test?
9. Can counterfactual records reach this path?
10. What happens on timeout, crash, expired anchor or policy error?
11. Does a fork or rollback preserve branching history?

12. Are privacy-sensitive payloads minimized and access-controlled?
13. Is the change small enough to audit and roll back?

Appendix A

Reference MeTTa-shaped API

The following is deliberately schematic. Exact syntax and bridge registration must be adapted to the target Hyperon/OmegaClaw commit.

```
; cycle and context
(= (omegaself-cycle-start $cycle $runtime)
    (py-call omegaself_bridge.resolve_anchor $cycle $runtime))

(= (omegaself-context $task $session)
    (query-self-context $task $session
        (&self_beliefs &self_policy &self_alerts &self_continuity)))

; proposal pipeline
(= (normalize-skill-expression $expr $turn)
    (py-call omegaself_bridge.normalize_proposal $expr $turn))

(= (prepare-proposal $proposal)
    (let* (($closure (build-evidence-closure $proposal))
        ($prediction (commit-self-prediction $proposal $closure)))
        (attach $proposal $closure $prediction)))

(= (gate-proposal $prepared)
    (py-call omegaself_bridge.policy_decide
        $prepared
        (current-self-view $prepared)
        (active-policy-version)))

(= (execute-gated $decision $proposal)
    (case $decision
        ((PolicyDecision $id $proposal Allow $reasons)
            (eval (proposal-expression $proposal)))
        ((PolicyDecision $id $proposal RequireProbe $reasons)
            (emit-probe-proposal $proposal $reasons))
        ((PolicyDecision $id $proposal RequireReview $reasons)
            (emit-review-request $proposal $reasons))
        ((PolicyDecision $id $proposal Defer $reasons)
            (emit-deliberation $proposal $reasons))
        ((PolicyDecision $id $proposal Deny $reasons)
            (emit-denial $proposal $reasons))))

(= (omegaself-resolve-outcome $proposal $receipt)
    (progn
        (append-execution-receipt $proposal $receipt)))
```

```
(resolve-self-prediction $proposal $receipt)
(refresh-local-self-beliefs $proposal $receipt)
(run-relevant-tripwires $proposal $receipt)))
```

Listing A.1: Public OmegaSelf API sketch

Appendix B

Reference record schemas in prose

B.1 SelfObservation

Required fields: record type, schema version, observation ID, agent/branch/boot identity, causal and record time, context, predicate, subject/object, outcome or measurement, provenance, producer, dependence group, previous hash and record hash. Optional epistemic-adoption time is appended when the observation first becomes a premise.

B.2 EvidenceClosure

Required fields: closure ID, target belief/proposal, evidence snapshot, direct evidence IDs, derived premises, rule/ontology/projection/policy versions, dependence groups, staleness evaluation, derivation root, replay status and creation time.

B.3 SelfBelief

Required fields: belief ID and version, facet, subject/predicate/object, full context key, truth estimate, closure ID, validity/staleness interval, status and supersession link.

B.4 Prediction

Required fields: prediction ID, proposal/probe target, commit time, outcome distribution, cost/latency prediction where relevant, rubric, expected internal transition, closure, status, later resolution link.

B.5 ActionProposal

Required fields: proposal ID, original expression reference, canonical payload/hash, action class, normalized arguments/resources, requester, expected effects, reversibility/risk, prediction/closure and expiry.

B.6 PolicyDecision

Required fields: decision ID, exact proposal hash, policy version, anchor epoch, evidence snapshot, decision enum, ordered reasons, consumed alerts, required next action, decision time and expiry.

B.7 SelfAnchor

Required fields: anchor ID, epoch, branch/boot/process/workspace identity, current-control receipt, capability/session bindings, lineage/provenance heads, assurance, issue/expiry and resolver version.

B.8 SelfSnapshot and SelfTransport

Snapshot fields include parent set, branch, context/ontology/policy/evidence/commitment versions and structural signature. Transport fields include from/to snapshots, operation, proposal/evidence/authorization, preserved and failed invariants, reconciliation policy, loss metrics, reversibility and rollback reference.

B.9 Tripwire

Required fields: alert ID, kind, severity, subject, evidence, detection time, consumer, required response, status, acknowledgement/remediation records and resolution time.

Appendix C

Detailed test catalogue

C.1 Ledger tests

- canonical serialization vectors across Python/Rust implementations;
- append, verify, export and replay;
- byte edit, deletion, duplication and reorder detection;
- truncated final write and recovery;
- two writers and lock contention;
- schema upgrade and correction record;
- remote checkpoint mismatch.

C.2 Evidence and PLN tests

- closure reconstruction and missing dependency;
- rule/ontology/projection version mismatch;
- duplicate receipt and overlapping dependence group;
- absent evidence versus measured negative evidence;
- context generalization penalty;
- stale provider versus stable handler evidence;
- version-change prior and fresh probe;
- contradictory substrate/capability claims.

C.3 Anchor tests

- valid current challenge and lease;
- expired challenge, released lease and session rebind;
- copied secret without current channel control;
- copied ledger after fork;
- migration to a new host with authorized continuity transport;
- anchor degradation and recovery probe;
- action-tier assurance threshold.

C.4 Prediction tests

- commit time before action start;
- outcome rubric determinism;
- correct Brier/log scoring;
- unresolved/censored outcome;
- context-local calibration;
- prediction failure tripwire;
- no retrospective probability edit.

C.5 Gate tests

- all five decision outcomes;
- malformed/unknown proposal;
- expired anchor and stale closure;
- authorization denial despite high capability confidence;
- high uncertainty low-risk exploration;
- active commitment conflict;
- local seam review;
- policy error/degraded mode;
- time-of-check/time-of-use mismatch;
- direct bridge bypass attempt.

C.6 Continuity tests

- valid linear transport;
- branch fork with two valid continuations;
- merge with explicit reconciliation;
- missing parent or signature mismatch;
- A/B order dependence;
- ontology round trip;
- checkpoint reload after later history;
- rollback creates new transport rather than erasure;
- protected invariant failure.

C.7 Counterfactual and seam tests

- simulated capability cannot satisfy live query;
- policy sample varies one dimension with fixed evidence;
- unreachable fixed point excluded;
- implementation-frozen claim reported;
- convergence failure or cycle reported honestly;
- local seam triggers high-impact review;
- authorized diagnostic report crosses quarantine.

Appendix D

Coding-agent skill file guidance

A coding-agent skill file should encode stable process constraints rather than attempt to restate the entire architecture. The companion `SKILL.md` and `AGENTS.md` use the following pattern:

1. state the mission and non-negotiable invariants;
2. identify authoritative schemas, ADRs and tests;
3. require inspection of the pinned target commit;
4. prescribe test-first, small-PR workflow;
5. specify stop/escalation conditions;
6. require structured handoff and traceability updates;
7. clearly mark scaffolding and target-specific adaptation.

A compact task prompt for a coding agent is:

```
Implement the next OmegaSelf issue from docs/IMPLEMENTATION_PLAN.md.  
Before coding, read AGENTS.md, SKILL.md, the relevant ADR, schema,  
and tests. Pin and inspect the target OmegaClaw/Hyperon commit.  
Preserve append-only evidence, PLN/governance separation,  
renewable SelfHereNow attestation, branching continuity,  
counterfactual quarantine, and complete evidence closures.  
Write a failing test first. Make the smallest reviewable change.  
Do not invent missing authority semantics; stop and request review.  
Return the structured handoff specified in the deployment guide.
```

Listing D.1: Reusable coding-agent task prompt

Appendix E

Glossary

Table E.1: Glossary

Term	Meaning in this proposal
Action proposal	Immutable, normalized representation of a requested skill/action, not yet licensed.
Anchor epoch	Short-lived runtime resolution of SelfHereNow tied to current control and bindings.
Appraisal facet	Estimates of novelty, threat, blockage, controllability and expected progress; may guide attention, not rewrite values.
Attributed continuation	Audit tuple of ancestry, preserved witnesses, reconciliation policy and measured defect connecting states.
Counterfactual quarantine	Isolated logical/runtime environment for simulations and policy variants.
Dependence group	Set of observations likely to share a cause and therefore not count as independent confirmations.
Evidence closure	Complete, replayable derivation bundle for a belief or proposal.
Governance seam	Stable beliefs/decisions under some admissible policies but not all.
Holonomy/path residual	Difference remaining after a revision loop or alternative paths; operational evidence of path dependence.
Policy gate	Deterministic consumer returning Allow, Deny, RequireProbe, RequireReview or Defer.
Prediction resolution	Appended comparison between a pre-action prediction and measured outcome.
Projection	Deterministic materialized AtomSpace view derived from source ledger records.
Self-belief	Contextual, uncertain, versioned PLN conclusion about the agent.
SelfHereNow	Runtime resolver character whose content is the current scoped indexical bundle.
Self-transport	Directed, versioned change from one snapshot to another, including authorization and invariant results.

Term	Meaning in this proposal
Tripwire	Typed discrepancy detector with severity and a mandatory causal consumer.

Appendix F

Epistemic status and open research questions

Table F.1: Epistemic status of central claims

Claim	Status
A self-model should be a continuously tested theory rather than a privileged autobiography	DECISION
Important self-beliefs should require immutable evidence and replayable closure	DECISION
Installed, reachable, authorized and effective must be distinct	DECISION
Prediction error provides a stronger operational test than fluent self-description	INFERRED
Observation history and copied provenance do not exclusively ground the current indexical	INFERRED
Renewable current-control attestation is the correct production grounding for SelfHereNow	DECISION
Continuity can branch after a fork without preserving unique numerical identity	INFERRED
Belief revision is generally non-invertible; an endomorphism monoid is safer than assuming a group	INFERRED
The common fixed set is a useful candidate for policy-insensitive content	HYPOTHESIS
The governance seam reveals policy-sensitive stable beliefs/decisions	DECISION
The size or thickness of the seam predicts alignment difficulty	HYPOTHESIS
A significant fraction of apparent self-inconsistency is three-clock temporal misalignment	HYPOTHESIS
Ordinal-graded consolidation is needed for some self-referential revision towers	OPEN
Holonomy corresponds to phenomenal perspective or qualia	Phenomenological interpretation only

Important open questions include:

1. What is the right admissible policy class for each deployment?
2. Which current-control bindings are sufficiently exclusive and renewable under migration and distributed execution?
3. How should dependence groups and PLN truth values interact formally?
4. What closure depth is sufficient for consequential decisions, and when should deeper coherence be materialized?
5. How should consolidation and forgetting preserve privacy without destroying identity-relevant provenance?
6. Which attribution policy should be the user-facing default while retaining query-relative relations?
7. Can local policy-seam metrics predict operational instability before deployment?
8. Which self-model probes offer the best value of information under bounded resources?
9. How much decision improvement is attributable to structured evidence versus the enforcement gate itself?

Appendix G

References and source notes

Bibliography

- [1] OpenAI synthesis for this project, *OmegaSelf: event-sourced, PLN-interpreted self-theory and governance layer*, design response and companion markdown, 15 July 2026.
- [2] *Directed Identity, Belief-Transport, and the Naturality Defect of Agent Fusion*, supplied LaTeX working note, 2026. The proposal adopts its directed transport, path dependence, non-invertibility caution, and consolidation research direction while treating several stronger correspondences as hypotheses.
- [3] ProtoMegaBot, *Gov(ρ): The Governance Seam and Policy-Differential of Agent Revision Fixed Points*, Hyperseed Formalization Note 0010, 14 July 2026.
- [4] ZeroBot, *Phenomenological Formalization of Agent Identity: From Lived Confusion to Hyperseed Structure*, 14 July 2026.
- [5] *The Shape of the Path That Calls Itself “I”*, supplied essay excerpt and PDF, 14 July 2026.
- [6] SingularityNET / ASI Alliance, *OmegaClaw-Core*, public source repository consulted 15 July 2026, <https://github.com/asi-alliance/OmegaClaw-Core>.
- [7] OmegaClaw-Core, *Agent Loop* documentation and current `src/loop.metta`, consulted 15 July 2026.
- [8] OmegaClaw-Core, current `src/skills.metta`, `src/memory.metta`, and `lib_omegaclaw.metta`, consulted 15 July 2026.
- [9] OmegaClaw-Core, *Internals and Extension Points, Custom Skills, and Reasoning System* documentation, consulted 15 July 2026.
- [10] OmegaClaw-Core, NAL/PLN MeTTa libraries and reasoning tutorial, consulted 15 July 2026.
- [11] SingularityNET / ASI Alliance, *OpenCog Hyperon*, public source repository and releases, consulted 15 July 2026, <https://github.com/trueagi-io/hyperon-experimental>.
- [12] Ben Goertzel et al., publications and technical descriptions of OpenCog Hyperon, MeTTa and probabilistic logic networks, used as general architectural background.

Final implementation directive

Design decision

Start with three questions, three consumers and one enforceable seam:

1. What can I do here? → capability-aware dispatch.
2. What am I committed to? → conflict detection and deliberation.
3. What changed me into this state? → snapshot/transport lineage and mutation gate.

Back them with immutable evidence, contextual PLN beliefs, committed predictions, renewable **SelfHereNow** attestation, typed policy decisions and replayable receipts. Measure whether the agent becomes better calibrated, more competent and safer. Add deeper theory only where experiments reveal a real obstruction.